M05 — Sorting & Order Statistics

Sources: CLRS Part II intro, Ch. 6 (Heapsort), Ch. 7 (Quicksort), Ch. 8 (Sorting in Linear Time), Ch. 9 (Medians and Order Statistics) · Skiena Ch. 4 (Sorting)

Big Idea

Sorting is the most-studied problem in algorithms for a reason: it is a **subroutine that makes dozens of other problems easy** (search, closest pair, uniqueness, mode, selection, convex hull), and it is the one nontrivial problem where we have a **matching upper and lower bound** — `Θ(n log n)` comparisons, no more and no less. This module is also where every major design paradigm shows up at once: incremental construction (insertion sort), divide and conquer (merge sort, quicksort), data-structure design (heapsort), and randomization (randomized quicksort, quickselect). The single most important structural insight: the `Ω(n log n)` barrier comes from a **counting argument over `n!` permutations** in the comparison model — and stepping *outside* that model (counting sort, radix sort, bucket sort) breaks it, at the price of assumptions about the keys. Months later, remember: *sort first, and the rest is usually easy*; *the lower bound is about `n!` leaves in a decision tree*; and *`Θ(n log n)` algorithms differ by constants that decide everything in practice*.

What You Should Be Able To Do After This Chapter

- Recognize when a problem becomes easy after sorting, and reach for it as a first move.
- Implement heapsort, quicksort (Lomuto and Hoare), merge sort, counting sort, radix sort, and quickselect in C++ from memory.
- Prove `BUILD-MAX-HEAP` is `Θ(n)` — not `O(n log n)` — via the `Σ h/2^h` argument.
- State and prove the `Ω(n lg n)` comparison lower bound from the decision-tree model.
- Give the *exact* condition under which two elements are compared in quicksort, and derive `E[X] = O(n lg n)` from it.
- Explain why randomized quicksort has no bad input, and why the `Θ(n²)` case is practically irrelevant.

- Say precisely what **stability** means, which algorithms have it, and why radix sort *requires* it.
- Choose between the linear-time sorts and know exactly what each assumes.
- Analyze quickselect's expected `O(n)` time via the "helpful partition" argument, and sketch median-of-medians.
- Give the constant-factor-optimal min-and-max algorithm (`3⌊n/2⌋` comparisons).

---

1. Why sorting matters

Skiena's applications list [§4.1, p.109]

> *Many important problems can be reduced to sorting, so we can use our clever `O(n log n)` algorithms to do work that might otherwise seem to require a quadratic algorithm.*

PROBLEM	POST-SORTING SOLUTION	TOTAL
Searching	binary search	`O(log n)` per query after `O(n log n)`
Closest pair (1-D)	the closest pair must be **adjacent**; one linear scan	`O(n log n)`
Element uniqueness	closest pair with gap 0 — scan adjacent pairs	`O(n log n)`
Finding the mode	identical items lump together; sweep and count	`O(n log n)`
Counting occurrences of `k`	binary search for `k−ε` and `k+ε`, subtract positions	`O(log n)` per query
Selection (`k`-th largest)	index position `k` directly	`O(1)` per query
Convex hull	sort by `x`, insert left to right; the rightmost point is always on the hull, and points it eliminates are neighbours of the previous insertion	`O(n log n)`

> *It is a rare application where the running time of sorting proves to be the bottleneck. …* ***Never be afraid to spend time sorting, provided you use an efficient sorting routine.***
>
> ***Take-Home Lesson:*** *Sorting lies at the heart of many algorithms.* ***Sorting the data is one of the first things any algorithm designer should try*** *in the quest for efficiency.*

CLRS's five reasons [Part II intro, p.158]

1. Some applications inherently need it (banks sorting checks by number).
2. It is a key subroutine (rendering layered graphics bottom-to-top).
3. It exhibits a rich set of design techniques.
4. **We can prove a nontrivial lower bound** — and the best upper bounds match it, so certain sorts are provably optimal. That lower bound then transfers to other problems.
5. Many engineering issues surface: memory hierarchy, satellite data, software environment.

The concrete argument for `n log n` [Skiena Fig. p.110]

N	N²/4	N LG N
10	25	33
100	2,500	664
1,000	250,000	9,965
10,000	25,000,000	132,877
100,000	2,500,000,000	1,660,960

> *You might survive using a quadratic-time algorithm even if* `n = 10,000`*, but the slow algorithm clearly gets ridiculous once* `n ≥ 100,000`*.*

Note that at `n = 10` and `n = 100` the quadratic one is *competitive or better* — which is exactly why hybrid sorts use insertion sort at the leaves.

Stop and Think: sorting vs hashing [Skiena §4.1, p.112]

Which of the sorting applications can hashing do as fast or faster in **expected** time?

PROBLEM	HASHING?
Searching	✓ **Better** — `O(1)` expected vs `O(log n)`
Closest pair	✗ Normal hash functions *scatter* keys, so similar values land in different buckets. Bucketing by numeric range helps, but you can't also bound the bucket size
Element uniqueness	✓ **Faster than sorting** — chain, then compare the expected-constant pairs within each bucket. Linear expected
Finding the mode	✓ Linear expected — sweep each bucket deleting duplicates
Finding the median	✗ The median could be in any bucket; no way to know how many items precede it
Convex hull	✗ Can't order points by `x`-coordinate

The pattern: hashing answers **equality** questions; sorting answers **order** questions. If your problem needs "which is bigger", hashing cannot help.

Stop and Think: set intersection [Skiena §4.1, p.112]

Two sets of sizes `m ≪ n`. Are they disjoint?

APPROACH	COST
Sort the big set, binary-search each small element	`O((n + m) log n)`
Sort the small set, binary-search each big element	**`O((n + m) log m)`** ← best
Sort both, then linear merge-scan	`O(n log n + m log m + n + m)`
Hash both, check collisions	`O(n + m)` expected — *"in practice, this may be the best solution"*

Sorting the small set wins because `log m < log n`, and `(n+m) log m` is asymptotically below `n log n`. This is linear when `m` is constant. A genuinely useful interview move.

---

2. Pragmatics: the questions to ask before you sort

[Skiena §4.2, p.113] — these are the questions an interviewer wants you to ask.

QUESTION	WHY IT MATTERS
Ascending or descending?	Application-specific; don't assume
Sorting the key or the whole record?	A mailing list sorted by name must keep names attached to addresses. Specify the key field and the record extent
What about equal keys?	Sometimes their relative order matters → **stability**. Sometimes you need a secondary key
Non-numeric data?	Is `Skiena` the same key as `skiena`? Is `Brown-Williams` before or after `Brown America`? Collation rules are genuinely complicated

Skiena's warning about ties, which people forget:

> *Certain efficient sort algorithms (such as quicksort) can run into **quadratic performance trouble** unless explicitly engineered to deal with large numbers of ties.*

Stability

> *Sorting algorithms that automatically leave items in the same relative order as in the original permutation are called **stable**. **Few fast algorithms are naturally stable.** Stability can be achieved for any sorting algorithm by **adding the initial position as a secondary key.***

ALGORITHM	STABLE?	IN PLACE?
Insertion sort	✓	✓
Merge sort	✓	✗ (`Θ(n)` buffer)
Heapsort	✗	✓
Quicksort	✗	✓ (but `Θ(log n)`–`Θ(n)` stack)
Counting sort	✓	✗
Radix sort	✓ (requires a stable digit sort)	✗
Bucket sort	✓ (if the per-bucket sort is)	✗

CLRS's generic stabilization [Ex. 8.3-2]: pair each key with its original index and break ties on the index. Costs `Θ(n)` extra space and `Θ(n)` extra time.

CLRS's distinctness trick [footnote, p.182], used throughout its analyses: convert `A[i]` to the ordered pair `(A[i], i)`. This makes all elements distinct at the cost of `Θ(n)` space and constant overhead — which is how the "assume distinct elements" assumption is discharged.

The library-function point

> *Any reasonable programming language has a built-in sort routine as a library function.* ***You are usually better off using this than writing your own routine.***

And Skiena's answer to "then why teach the algorithms?":

> *The answer is that the underlying design techniques are very important for other algorithmic problems you are likely to encounter.*

C++ specifics (outside/engineering context, since the books use C's `qsort`): use `std::sort` (introsort — quicksort + heapsort fallback + insertion-sort leaves; `O(n log n)` worst case, **not stable**), `std::stable_sort` (merge sort; falls back to in-place merge if allocation fails), `std::partial_sort` (heap-based, top-`k` in `O(n log k)`), and `std::nth_element` (introselect, `O(n)` expected — this is quickselect).

---

3. Heaps

Unified Understanding

> *Heaps work by maintaining a* ***partial order*** *on the set of elements that is* ***weaker than the sorted order*** *(so it can be efficient to maintain) yet* ***stronger than random order*** *(so the minimum element can be quickly identified). [Skiena §4.3.1, p.116]*

That one sentence is the entire justification for the data structure.

Definition [CLRS §6.1, p.161]

A **binary heap** is an array viewed as a **nearly complete binary tree**: completely filled on all levels except possibly the lowest, which is filled from the left up to a point.

`A.heap-size` records how many of `A[1..n]` are actually in the heap.

```
PARENT(i) = ⌊i/2⌋       LEFT(i) = 2i       RIGHT(i) = 2i + 1
```

> *On most computers, `LEFT` can compute `2i` in one instruction by shifting left one bit; `RIGHT` shifts and adds 1; `PARENT` shifts right one bit. Good implementations often implement these as macros or inline procedures.*

The heap property:

- **Max-heap:** `A[PARENT(i)] ≥ A[i]` for every node `i` other than the root. Largest at the root.
- **Min-heap:** `A[PARENT(i)] ≤ A[i]`. Smallest at the root.

Heapsort uses max-heaps; priority queues typically use min-heaps.

Height. An `n`-element heap has height `⌊lg n⌋`, so all operations are `O(lg n)`. [CLRS Ex. 6.1-2]

Why the implicit array representation works — and its cost [Skiena §4.3.1]

> *The heap is a slick data structure that enables us to represent binary trees **without using any pointers**. We store data as an array of keys, and use the position of the keys to implicitly play the role of the pointers.*

The catch: if the tree were sparse, all missing internal nodes still occupy space.

> *Space efficiency thus demands that we not allow holes in our tree... If we did not enforce these structural constraints, we might need an array of size $2^n - 1$ to store n elements: consider a right-going twig using positions 1, 3, 7, 15, 31...*

> *This implicit representation saves memory, but is **less flexible** than using pointers. We cannot store arbitrary tree topologies without wasting large amounts of space. We cannot move subtrees around by just changing a single pointer, only by explicitly moving all the elements. **This loss of flexibility explains why we cannot use this idea to represent binary search trees, but it works just fine for heaps.***

Stop and Think: searching a heap [Skiena §4.3.1, p.118]

Problem. How do we efficiently search for a key `k` in a heap?

Solution. *"We can't."* Binary search doesn't apply — a heap is not a BST. We know essentially nothing about the relative order of the `n/2` leaves, so linear search is unavoidable.

This is the single most common heap misconception. A heap gives you the extremum in `O(1)`; it gives you *nothing* about anything else.

Algorithm: MAX-HEAPIFY (sift down / bubble down)

Problem. `A[i]` may violate the max-heap property, but the subtrees rooted at `LEFT(i)` and `RIGHT(i)` are max-heaps. Restore the property at `i`.

Core intuition. Let `A[i]` "float down": swap it with its largest child, and recurse there.

Pseudocode

```
MAX-HEAPIFY(A, i)
1  l = LEFT(i);  r = RIGHT(i)
2  if l ≤ A.heap-size and A[l] > A[i]:  largest = l
3  else:                               largest = i
4  if r ≤ A.heap-size and A[r] > A[largest]:  largest = r
5  if largest ≠ i
6      exchange A[i] with A[largest]
7      MAX-HEAPIFY(A, largest)
```

→ **C++ implementation:** A1 MAX-HEAPIFY

Complexity. Each child's subtree has at most `2n/3` nodes (worst case: the bottom level is exactly half full) [CLRS Ex. 6.2-2]:

```
T(n) ≤ T(2n/3) + θ(1)   →   master case 2   →   O(lg n)
```

Better characterization: **`O(h)` for a node of height `h`** — this is what makes the linear build-time analysis work.

Algorithm: BUILD-MAX-HEAP — and why it is `θ(n)`, not `θ(n log n)`

```
BUILD-MAX-HEAP(A, n)
1  A.heap-size = n
2  for i = ⌊n/2⌋ downto 1
3      MAX-HEAPIFY(A, i)
```

→ **C++ implementation:** A2 BUILD-MAX-HEAP

Why start at `⌊n/2⌋` and go down? The elements `A[⌊n/2⌋+1 .. n]` are all **leaves** [Ex. 6.1-8], hence already 1-element heaps. And you must go *downward* in index so that when you heapify node `i`, both its children are already heap roots [Ex. 6.3-3].

Loop invariant. *At the start of each iteration, each node `i+1, i+2, …, n` is the root of a max-heap.*

- **Initialization:** `i = ⌊n/2⌋`; nodes `⌊n/2⌋+1 … n` are leaves, trivially max-heaps.
- **Maintenance:** node `i`'s children are numbered higher than `i`, so by the invariant they are max-heap roots — exactly `MAX-HEAPIFY`'s precondition. The call makes `i` a max-heap root and preserves the others.
- **Termination:** `i = 0`; every node `1..n` is a max-heap root, in particular node 1. ∎

The complexity. The naive bound: `O(n)` calls × `O(lg n)` each = `O(n lg n)`. **Correct, but not tight.**

Two facts:

1. `MAX-HEAPIFY` on a node of height `h` costs `O(h)`.
2. There are at most `⌈n/2^{h+1}⌉` nodes of height `h` [Ex. 6.3-4].

```
Σ_{h=0}^{⌊lg n⌋} ⌈n/2^{h+1}⌉ · ch  ≤  Σ_{h=0}^{⌊lg n⌋} (n/2^h) · ch
                                    =  cn · Σ_{h=0}^{⌊lg n⌋} h/2^h
                                    ≤  cn · Σ_{h=0}^{∞} h/2^h
                                    =  cn · (1/2)/(1 − 1/2)²
[CLRS eq. A.11]
                                    =  2cn  =  O(n)
```

Hence a max-heap can be built from an unordered array in linear time.

Skiena's gloss on the same sum [§4.3.4, p.122]:

> *Since this sum is not quite a geometric series, we can't apply the usual identity, but **rest assured that the puny contribution of the numerator (`h`) is crushed by the denominator (`2^h`)**. The series quickly converges to linear.*

And the honest assessment:

> *Does it matter that we can construct heaps in linear time instead of `O(n log n)`? **Not really.** The construction time did not dominate the complexity of heapsort. Still, it is an impressive display of the power of careful analysis, and the free lunch that geometric series convergence can sometimes provide.*

The contrast worth remembering [CLRS Problem 6-1]: building by `n` successive `MAX-HEAP-INSERT` calls takes `Θ(n lg n)` in the worst case — *and produces a different heap*. **Bottom-up build (`Θ(n)`) ≠ repeated insertion (`Θ(n log n)`).** The reason is the shape of the work: bottom-up does most of its work on short subtrees, insertion does most of its work on tall ones.

Algorithm: Heapsort

Core intuition [Skiena §4.3, p.116] — and this framing is the best one:

> ***The name typically given to this algorithm, heapsort, obscures the fact that the algorithm is nothing but an implementation of selection sort using the right data structure.***

Selection sort repeatedly extracts the minimum from an unsorted array: `O(n)` to find it, `O(1)` to remove. Replace the array with a heap and both become `O(log n)`. `O(n²) → O(n log n)`, purely from the data structure.

Pseudocode

```
HEAPSORT(A, n)
1  BUILD-MAX-HEAP(A, n)
2  for i = n downto 2
3      exchange A[1] with A[i]        // largest goes to its final
position
4      A.heap-size = A.heap-size - 1  // discard it from the heap
5      MAX-HEAPIFY(A, 1)              // restore the property at
the root
```

→ **C++ implementation:** A3 HEAPSORT

Loop invariant [Ex. 6.4-2]: *At the start of each iteration, `A[1..i]` is a max-heap containing the `i` smallest elements of `A[1..n]`, and `A[i+1..n]` contains the `n−i` largest, sorted.*

Complexity. `O(n)` for the build + `(n−1)` calls of `O(lg n)` = **`O(n lg n)`**, worst case. Best case is also `Ω(n lg n)` for distinct elements [Ex. 6.4-5]. In place, `O(1)` auxiliary.

C++ Implementation

```cpp
#include <vector>
#include <utility>

namespace detail {

// Sift v[i] down within v[0 .. size-1]. 0-indexed: children are 2i+1 and 2i+2.
// Iterative (CLRS Ex. 6.2-6) -- avoids recursion overhead entirely.
template <typename T>
void siftDown(vector<T>& v, int i, int size) {
    T key = move(v[i]);
    while (true) {
        const int l = 2 * i + 1;
        if (l >= size) break;
        // Pick the larger child.
        const int child = (l + 1 < size && v[l] < v[l + 1]) ? l + 1 : l;
        if (!(key < v[child])) break;          // key already dominates
        v[i] = move(v[child]);             // move child up (one write, not a swap)
        i = child;
    }
    v[i] = move(key);
}

}  // namespace detail

// Heapsort: O(n log n) worst case, O(1) auxiliary space, NOT stable.
template <typename T>
void heapSort(vector<T>& v) {
    const int n = static_cast<int>(v.size());
    // Build phase: Theta(n). Start at the last internal node.
    for (int i = n / 2 - 1; i >= 0; --i) detail::siftDown(v, i, n);
    // Sort phase: n-1 extractions.
    for (int i = n - 1; i > 0; --i) {
        swap(v[0], v[i]);
        detail::siftDown(v, 0, i);
```

```
    }
}
```

Implementation notes

- **0-indexed children are `2i+1`, `2i+2`; the parent is `(i−1)/2`.** CLRS and Skiena both use 1-indexing where children are `2i`, `2i+1`. Pick one and be consistent — mixing them is the classic heap bug.
- **The last internal node is `n/2 − 1`** (0-indexed), equivalently `⌊n/2⌋` (1-indexed).
- **`siftDown` hoists the key out and does one write per level** instead of a 3-assignment swap. Roughly a 30% improvement, and the same trick CLRS suggests for `MAX-HEAP-INCREASE-KEY` [Ex. 6.5-8] and that insertion sort uses.
- **Heapsort is cache-hostile.** Its access pattern jumps by powers of two, defeating prefetching. This is the main reason `std::sort`'s introsort uses quicksort first and only *falls back* to heapsort when the recursion gets too deep.
- Prefer `std::make_heap` / `std::sort_heap` / `std::priority_queue` in practice.

Common bugs

- Mixing 1-indexed and 0-indexed child formulas.
- Starting the build loop at `n/2` instead of `n/2 − 1` (0-indexed) — off by one, misses a node.
- Forgetting to shrink the heap size in the sort phase, so extracted maxima get re-heapified.
- Sifting **up** in the build phase (that's the `Θ(n log n)` algorithm).
- Assuming heapsort is stable.

Recognition pattern

Heapsort when you need **guaranteed `O(n log n)` *and* `O(1)` space** and don't need stability. Otherwise the heap itself is the point — see priority queues next.

---

4. Priority Queues

Operations [CLRS §6.5, p.173]

MAX-PRIORITY QUEUE	MIN-PRIORITY QUEUE	COST
`INSERT(S, x, k)`	`INSERT`	`O(lg n)`
`MAXIMUM(S)`	`MINIMUM`	`θ(1)`
`EXTRACT-MAX(S)`	`EXTRACT-MIN`	`O(lg n)`
`INCREASE-KEY(S, x, k)`	`DECREASE-KEY`	`O(lg n)`

Applications named by CLRS: max-PQ for **job scheduling** on a shared computer (`EXTRACT-MAX` picks the next job, `INSERT` adds one); min-PQ for **event-driven simulation** (events keyed by occurrence time, since simulating one event can spawn future ones). And `DECREASE-KEY` specifically for **Prim's MST** (M14) and **Dijkstra** (M15).

The handle problem — an implementation detail that matters

`DECREASE-KEY` needs to find element `x` in the heap. But heap elements **move** during operations. Two solutions [CLRS §6.5, p.174]:

APPROACH	HOW	TRADE-OFF
Handles	Store the heap index inside the application object (opaque to the application). Every relocation updates the handle	`O(1)` overhead per access; requires modifying the application objects
External map	The PQ keeps a hash table from object → array index	Nothing added to application objects; `O(1)` expected but `θ(n)` worst-case lookup, plus maintenance cost

Why this matters in practice: `std::priority_queue` supports **neither** — it has no `decrease-key`. The standard workaround in Dijkstra is the **lazy-deletion pattern**: push a new `(newDist, vertex)` pair and skip stale entries on pop. See M15.

`INCREASE-KEY` and `INSERT`

```
MAX-HEAP-INCREASE-KEY(A, x, k)
1  if k < x.key:  error "new key is smaller than current key"
2  x.key = k
3  find the index i where object x occurs
4  while i > 1 and A[PARENT(i)].key < A[i].key
```

```
5       exchange A[i] with A[PARENT(i)], updating the object→index
mapping
6       i = PARENT(i)

MAX-HEAP-INSERT(A, x, n)
1   if A.heap-size == n:  error "heap overflow"
2   A.heap-size = A.heap-size + 1
3   k = x.key;  x.key = −∞
4   A[A.heap-size] = x;  map x to that index
5   MAX-HEAP-INCREASE-KEY(A, x, k)
```

→ **C++ implementation:** A4 MAX-HEAP-INSERT and MAX-HEAP-INCREASE-KEY

The `while` loop of `INCREASE-KEY` is *"reminiscent of the insertion loop of `INSERTION-SORT`"* — bubble up until the parent dominates.

Two exercise answers worth internalizing:

- **Why set `x.key = −∞` first (Ex. 6.5-5)?** Because `INCREASE-KEY` errors out if the new key is smaller than the current one. Placing garbage at the new leaf could trigger that check spuriously.
- **Why can't `MAX-HEAPIFY` replace the bubble-up loop (Ex. 6.5-6)?** `MAX-HEAPIFY` moves a value **down**; an increased key needs to move **up**. Wrong direction.

C++ Implementation — indexed min-priority queue with decrease-key

```
#include <vector>
#include <utility>
#include <stdexcept>
#include <limits>

// Min-heap over ids 0..capacity-1, supporting decreaseKey. This is
the
// structure Dijkstra and Prim actually want. All ops O(log n).
class IndexedMinPQ {
public:
    explicit IndexedMinPQ(int capacity)
        : pos_(capacity, -1), key_(capacity, numeric_limits<long
long>::max()) {}
```

```
    bool empty() const { return heap_.empty(); }
    bool contains(int id) const { return pos_[id] != -1; }
    long long keyOf(int id) const { return key_[id]; }

    void push(int id, long long k) {
        if (contains(id)) { decreaseKey(id, k); return; }
        key_[id] = k;
        heap_.push_back(id);
        pos_[id] = static_cast<int>(heap_.size()) - 1;
        siftUp(pos_[id]);
    }

    // Lower id's key to k. No-op if k is not an improvement.
    void decreaseKey(int id, long long k) {
        if (!contains(id) || k >= key_[id]) return;
        key_[id] = k;
        siftUp(pos_[id]);
    }

    int popMin() {
        if (heap_.empty()) throw underflow_error("empty priority
queue");
        const int top = heap_.front();
        swapNodes(0, static_cast<int>(heap_.size()) - 1);
        heap_.pop_back();
        pos_[top] = -1;
        if (!heap_.empty()) siftDown(0);
        return top;
    }

private:
    vector<int> heap_;        // heap_[i] = id at heap position i
    vector<int> pos_;         // pos_[id] = heap position of id, or
-1
    vector<long long> key_;

    void swapNodes(int i, int j) {
        swap(heap_[i], heap_[j]);
        pos_[heap_[i]] = i;                  // the handle update
CLRS describes
        pos_[heap_[j]] = j;
```

```
    }
    void siftUp(int i) {
        while (i > 0) {
            const int p = (i - 1) / 2;
            if (key_[heap_[p]] <= key_[heap_[i]]) break;
            swapNodes(i, p);
            i = p;
        }
    }
    void siftDown(int i) {
        const int n = static_cast<int>(heap_.size());
        while (true) {
            const int l = 2 * i + 1;
            if (l >= n) break;
            int c = (l + 1 < n && key_[heap_[l + 1]] <
key_[heap_[l]]) ? l + 1 : l;
            if (key_[heap_[i]] <= key_[heap_[c]]) break;
            swapNodes(i, c);
            i = c;
        }
    }
};
```

War Story: Give me a Ticket on an Airplane

[Skiena §4.4, p.125] — the best illustration of a priority queue as a *lazy generator*.

The problem. Find the cheapest legal airfare from `x` to `y`. Skiena's opening move — model as a graph, run Dijkstra — got laughed out of the room, because airline pricing has millions of fares, changing several times daily, governed by an industry-wide kludge of rules with no consistent logic. *(His favourite: rules that apply only to Malawi. "Accurately pricing any air itinerary requires at least implicit checks to ensure the trip doesn't take us through Malawi.")*

The real problem. ~100 fares for LAX→ORD, ~100 for ORD→JFK. The cheapest LAX–ORD fare may be *incompatible* with the cheapest ORD–JFK fare, and legality is only decidable by calling an expensive black-box routine. So: **evaluate all `m×n` combinations in increasing order of total cost, stopping at the first legal one.**

Constructing and sorting all `m×n` pairs is `O(nm log(nm))` — and wasteful, since the first pair might be all you need.

The insight. Both fare lists come out of the database **already sorted**. Therefore `(i+1, j)` and `(i, j+1)` are never cheaper than `(i, j)`. So:

- Seed a priority queue keyed by total cost with just `(1, 1)`.
- Pop the cheapest pair `(i, j)`; test it.
- If illegal, push its two successors `(i+1, j)` and `(i, j+1)`.

The duplicate problem. `(x, y)` gets generated twice — from `(x−1, y)` and from `(x, y−1)`. Guard with a hash set (or, the cleaner fix: only push `(i, j+1)` always and `(i+1, j)` only when `j == 1`).

> *We will never have more than `n` active pairs in our data structure, since there can only be one pair for each distinct value of the first coordinate.*

> *The **best-first evaluation** inherent in our priority queue enabled the system to stop as soon as it found the provably cheapest fare. This proved to be fast enough to provide interactive response to the user. **That said, I never noticed airline tickets getting cheaper as a result.***

Recognition pattern — this is a very common interview problem. "k smallest pairs from two sorted arrays", "k-th smallest in a sorted matrix", "merge k sorted lists", "k smallest sums" are all this pattern: **a priority queue as a lazy frontier over a partially ordered generation graph.** Skiena's footnote flags that whether all `n×m` sums can be sorted faster than `nm` arbitrary integers is a famous open problem (the **X + Y sorting** problem).

C++ Implementation — k smallest pairs

```cpp
#include <vector>
#include <queue>
#include <utility>
#include <cstdint>

// The k cheapest sums a[i] + b[j], a and b sorted ascending.
// O(k log k) time, O(k) space -- never materializes the n*m pairs.
vector<pair<int,int>>
kSmallestPairs(const vector<int>& a, const vector<int>& b, int k) {
    vector<pair<int,int>> out;
    if (a.empty() || b.empty() || k <= 0) return out;

    struct Node { int64_t sum; int i, j; };
```

```
    auto worse = [](const Node& x, const Node& y) { return x.sum >
y.sum; };
    priority_queue<Node, vector<Node>, decltype(worse)> pq(worse);

    pq.push({static_cast<int64_t>(a[0]) + b[0], 0, 0});
    while (!pq.empty() && static_cast<int>(out.size()) < k) {
        const Node cur = pq.top(); pq.pop();
        out.push_back({cur.i, cur.j});
        // Push (i, j+1) always; push (i+1, j) only from the j == 0
column.
        // This generates every pair exactly once -- no duplicate
check needed.
        if (cur.j + 1 < static_cast<int>(b.size()))
            pq.push({static_cast<int64_t>(a[cur.i]) + b[cur.j + 1],
cur.i, cur.j + 1});
        if (cur.j == 0 && cur.i + 1 < static_cast<int>(a.size()))
            pq.push({static_cast<int64_t>(a[cur.i + 1]) + b[0],
cur.i + 1, 0});
    }
    return out;
}
```

Stop and Think: `k`-th smallest vs `x`, in `O(k)` [Skiena §4.3.4, p.123]

Problem. Given an array min-heap and a real `x`, decide whether the `k`-th smallest element is `≥ x`, in `O(k)` **independent of heap size**.

Two natural but too-slow ideas: `k` extract-mins is `O(k log n)`; scanning the first `k` levels is `O(min(n, 2^k))`.

The `O(k)` solution. Recurse from the root, only descending below nodes whose value is `< x`:

```
// Returns how many of the k needed "small" elements remain
unfound.
// Result 0  <=>  at least k elements are < x  <=>  kth smallest is
< x.
int heapCompare(const vector<int>& heap, int i, int count, int x) {
    if (count <= 0 || i >= static_cast<int>(heap.size())) return
count;
    if (heap[i] < x) {
        count = heapCompare(heap, 2 * i + 1, count - 1, x);
        count = heapCompare(heap, 2 * i + 2, count, x);
    }
    return count;
}
```

Why `O(k)`: we only look at the children of nodes with value `< x`, and there are at most `k` such nodes before `count` hits 0. Each has 2 children, so **at most `2k + 1` nodes are visited**. If the root is `≥ x`, we stop immediately — the root is the minimum.

---

5. Algorithm: Quicksort

Problem

Sort `A[p..r]` in place.

The three steps [CLRS §7.1, p.183]

- **Divide** by **partitioning** `A[p..r]` into `A[p..q-1]` (the low side, all `≤` pivot) and `A[q+1..r]` (the high side, all `>` pivot), computing `q` as part of the process.
- **Conquer** by recursively sorting both sides.
- **Combine** by **doing nothing.** *"The entire subarray `A[p..r]` cannot help but be sorted!"*

Two things partitioning buys you [Skiena §4.6, p.130]:

1. The pivot ends up in **exactly** its final sorted position.
2. **No element ever crosses the pivot again.** So the two sides are independent subproblems.

Pseudocode

```
QUICKSORT(A, p, r)
1  if p < r
2      q = PARTITION(A, p, r)
3      QUICKSORT(A, p, q − 1)
4      QUICKSORT(A, q + 1, r)

PARTITION(A, p, r)                          // Lomuto
1  x = A[r]                                 // the pivot
2  i = p − 1                                // highest index into the
low side
3  for j = p to r − 1
4      if A[j] ≤ x
5          i = i + 1
6          exchange A[i] with A[j]
7  exchange A[i + 1] with A[r]              // pivot into place
8  return i + 1
```

→ **C++ implementation:** A5 QUICKSORT and PARTITION

Correctness — the partition loop invariant [CLRS §7.1, p.184]

> *At the beginning of each iteration of the loop of lines 3–6, for any array index* `k`*:*
>
> 1. *if* `p ≤ k ≤ i`*, then* `A[k] ≤ x` *(**low side**);*
> 2. *if* `i+1 ≤ k ≤ j−1`*, then* `A[k] > x` *(**high side**);*
> 3. *if* `k = r`*, then* `A[k] = x` *(**the pivot**).*

Four regions: `[p..i]` are `≤ x`; `[i+1..j−1]` are `> x`; `[j..r−1]` are **unexamined**; `A[r] = x`.

- **Initialization.** `i = p−1`, `j = p`: both ranges are empty; line 1 gives condition 3.
- **Maintenance.** If `A[j] > x`: only `j` increments, so condition 2 now covers `A[j−1]`. If `A[j] ≤ x`: increment `i`, swap `A[i]↔A[j]`, increment `j`. After the swap `A[i] ≤ x` (condition 1) and `A[j−1] > x` (it was the element previously at `A[i]`, which by the invariant was `> x`).
- **Termination.** `j = r`; `A[j..r−1]` is empty, so all elements are classified. Lines 7–8 swap the pivot into position `i+1`. ∎

Stronger fact worth noting: after line 2 of `QUICKSORT`, `A[q]` is **strictly** less than every element of `A[q+1..r]`. That's what makes the recursion exclude `q`.

`PARTITION` runs in `Θ(n)` on `n = r − p + 1` elements.

Performance — where the height comes from

Quicksort runs in `O(n · h)` where `h` is the recursion-tree height. Everything depends on the pivot.

CASE	RECURRENCE	RESULT
Worst — split `n−1` / `0` every time	`T(n) = T(n−1) + Θ(n)`	**`Θ(n²)`** (arithmetic series)
Best — even split	`T(n) = 2T(n/2) + Θ(n)`	`Θ(n lg n)`
9-to-1 split every time	`T(n) = T(9n/10) + T(n/10) + Θ(n)`	**`O(n lg n)`**

The 9-to-1 result is the important one [CLRS §7.2, p.188]. Every level costs `≤ n`; the shallowest leaf is at depth `log₁₀ n` and the deepest at `log_{10/9} n` — both `Θ(lg n)`.

> *Even a **99-to-1** split yields an `O(n lg n)` running time. In fact, **any split of constant proportionality** yields a recursion tree of depth `Θ(lg n)` where the cost at each level is `O(n)`. … **The ratio of the split affects only the constant hidden in the `O`-notation.***

And the worst case is the sorted array — the one case insertion sort handles in `O(n)`.

Why the average is close to the best — two arguments

CLRS's "bad split absorbed by good split" [Fig. 7.5, p.190]. Suppose good and bad splits alternate. A bad split at the root (sizes `0` and `n−1`, cost `Θ(n)`) followed by a good split of the `n−1` piece (sizes `(n−1)/2 − 1` and `(n−1)/2`, cost `Θ(n−1)`) produces the same three subarrays as a *single* good split — at a combined cost of `Θ(n) + Θ(n−1) = Θ(n)`.

> *Intuitively, the `Θ(n−1)` cost of the bad split can be **absorbed into** the `Θ(n)` cost of the good split, and the resulting split is good. Thus, the running time when levels alternate between good and bad splits is like the running time for good splits alone: still `O(n lg n)`, but with a slightly larger constant.*

Skiena's "good enough pivot" [§4.6.1, p.132]. Call a pivot *good enough* if it ranks between `n/4` and `3n/4`. Exactly **half** of all elements qualify, so a random pivot is good enough with probability `1/2`. The worst good-enough pivot leaves a partition of `3n/4`, so the deepest path is `n, (3/4)n, (3/4)²n, …, 1`:

```
(3/4)^h · n = 1   ⟹   h = log_{4/3} n
```

> *More careful analysis shows the average height after `n` insertions is approximately `2 ln n`. Since `2 ln n ≈ 1.386 lg n`, this is **only 39% taller** than a perfectly balanced binary tree.*

That `1.386×` figure is the number to remember — it also gives the average height of a randomly-built BST (M08).

Randomized quicksort

```
RANDOMIZED-PARTITION(A, p, r)
1  i = RANDOM(p, r)
2  exchange A[r] with A[i]
3  return PARTITION(A, p, r)
```

→ **C++ implementation:** A6 RANDOMIZED-PARTITION

CLRS notes that although you *could* pre-shuffle the whole array (as with `RANDOMIZED-HIRE-ASSISTANT`), *"a different randomization technique yields a simpler analysis"* — randomize the pivot instead.

The rigorous expected-time analysis — the argument to know

Lemma 7.1. Quicksort's running time is `O(n + X)` where `X` is the number of **element comparisons**.
Why: each `PARTITION` call removes its pivot from all future calls, so there are $\leqslant$ `n` calls to `PARTITION` and $\leqslant$ `2n` to `QUICKSORT`. Outside the `for` loop each call is `O(1)`, giving `O(n)`; inside, every iteration is one comparison, giving `X`.

Reindex by sorted position: call the elements $z_1 < z_2 < \cdots < z_n$, and write $Z_{ij} = \{z_i, z_{i+1}, \ldots, z_j\}$.

Lemma 7.2 — the characterization.

> *z_i is compared with z_j ($i < j$) **if and only if one of them is chosen as a pivot before any other element of z_{ij}**. Moreover, no two elements are ever compared twice.*

Proof. Consider the first $x \in z_{ij}$ chosen as a pivot. If $z_i < x < z_j$, then z_i and z_j fall on **opposite sides** of the partition and can never be compared. If $x = z_i$ or $x = z_j$, `PARTITION` compares it with every other element of z_{ij} — including the other one. And a pivot is removed from all future comparisons. ∎

The illustrating example: sorting `1..10`, first pivot `7`. Sets become `{1..6}` and `{8,9,10}`. `7` and `9` **are** compared (7 is the first pivot from $z_{7,9}$). `2` and `9` are **never** compared (the first pivot from $z_{2,9}$ is `7`, strictly between them).

Lemma 7.3 — the probability.

$$\Pr\{z_i \text{ compared with } z_j\} = 2/(j - i + 1)$$

Why: all of z_{ij} stays together through the recursion until some $x \in z_{ij}$ is chosen as a pivot. At that moment each of the $|z_{ij}| = j - i + 1$ elements is **equally likely** to be x. Two of them (z_i, z_j) give a comparison, and the events are mutually exclusive.

Theorem 7.4. $E[X] = O(n \lg n)$.

$$\begin{aligned}
E[X] &= \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \Pr\{z_i \text{ compared with } z_j\} && \text{[indicators + linearity]} \\
&= \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} 2/(j - i + 1) \\
&= \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} 2/(k + 1) && [k = j - i] \\
&< \sum_{i=1}^{n-1} \sum_{k=1}^{n} 2/k \\
&= \sum_{i=1}^{n-1} O(\lg n) && \text{[harmonic series]} \\
&= O(n \lg n) && \blacksquare
\end{aligned}$$

Note how little machinery this needs: indicator random variables (M04) plus the harmonic series (M02). This is the model for essentially every randomized-algorithm analysis.

Worst case is still $\Theta(n^2)$ [CLRS §7.4.1], via substitution on $T(n) = \max\{T(q) + T(n-1-q) : 0 \le q \le n-1\} + \Theta(n)$, using $q^2 + (n-1-q)^2 \le (n-1)^2$.

C++ Implementation

```
#include <vector>
#include <random>
#include <algorithm>
#include <utility>

namespace detail {

// Hoare-style 3-way partition (Dutch National Flag). Returns [lt,
gt] such that
//   v[lo..lt-1]  < pivot
//   v[lt..gt]   == pivot
//   v[gt+1..hi]  > pivot
// The 3-way split is what makes quicksort O(n) on all-equal input
rather than O(n^2).
template <typename T>
pair<int,int> partition3(vector<T>& v, int lo, int hi, const T&
pivot) {
    int lt = lo, i = lo, gt = hi;
    while (i <= gt) {
        if      (v[i] < pivot) swap(v[lt++], v[i++]);
        else if (pivot < v[i]) swap(v[i], v[gt--]);   // do NOT
advance i
        else                   ++i;
    }
    return {lt, gt};
}

template <typename T, typename RNG>
void quickSortRec(vector<T>& v, int lo, int hi, RNG& rng) {
    while (lo < hi) {
        if (hi - lo < 16) {                       // coarsen the
leaves (CLRS Ex. 7.4-5)
            for (int i = lo + 1; i <= hi; ++i) {
                T key = move(v[i]);
                int j = i - 1;
                while (j >= lo && key < v[j]) { v[j + 1] =
move(v[j]); --j; }
                v[j + 1] = move(key);
            }
            return;
```

```
        }
        uniform_int_distribution<int> pick(lo, hi);
        const T pivot = v[pick(rng)];             // copy: v is
about to be permuted
        auto [lt, gt] = partition3(v, lo, hi, pivot);

        // Recurse on the SMALLER side, loop on the larger (CLRS
Problem 7-5).
        // Guarantees O(log n) stack depth even in the worst case.
        if (lt - lo < hi - gt) { quickSortRec(v, lo, lt - 1, rng);
lo = gt + 1; }
        else                   { quickSortRec(v, gt + 1, hi, rng);
hi = lt - 1; }
    }
}

}  // namespace detail

// Randomized 3-way quicksort. O(n log n) expected, O(n^2) worst
case (astronomically
// unlikely), O(log n) stack, in place, NOT stable.
template <typename T>
void quickSort(vector<T>& v) {
    if (v.size() < 2) return;
    mt19937 rng(random_device{}());
    detail::quickSortRec(v, 0, static_cast<int>(v.size()) - 1,
rng);
}
```

Implementation notes

- **3-way partitioning is not optional in production.** Plain Lomuto on an all-equal array is `Θ(n²)` [CLRS Ex. 7.2-2]. This is CLRS Problem 7-2 and the reason `std::sort` handles duplicates gracefully.
- **Tail-recursion elimination + recurse-on-the-smaller-side** bounds the stack at `Θ(lg n)` [CLRS Problem 7-5]. Without it, the worst case uses `Θ(n)` stack and can blow it.
- **Insertion-sort cutoff at ~16.** The modified algorithm runs in `O(nk + n lg(n/k))` expected [Ex. 7.4-5]. `k` in the 10–30 range is empirically right.
- **The pivot is copied, not referenced** — `v` is permuted underneath, so a

reference would dangle semantically.

- **Median-of-3 pivots** [CLRS Problem 7-6] improve the constant but *"affect only the constant factor"* in the `Ω(n lg n)` running time. Widely used anyway.
- **McIlroy's "killer adversary"** [chapter notes] constructs an input that makes virtually *any* deterministic quicksort implementation run in `θ(n²)` — by answering comparisons adaptively. Only true randomization defeats it.

Common bugs

- Advancing `i` in the `pivot < v[i]` branch of the 3-way partition. The element swapped down from `gt` is unexamined.
- Using a **reference** to the pivot element instead of a copy.
- Recursing on `[lo, q]` instead of `[lo, q−1]` — infinite recursion.
- No 3-way handling with many duplicates → quadratic.
- Choosing `v[hi]` as the pivot and then sorting nearly-sorted data.

Recognition pattern

Quicksort is the **default** in-memory sort: in place, excellent cache behaviour, small constants. Use it unless you need **stability** (→ merge sort) or a **worst-case guarantee** (→ heapsort, or introsort which combines them).

Is quicksort really quick? [Skiena §4.6.3, p.135]

> *How can we compare two `θ(n log n)` algorithms to decide which is faster? … Unfortunately, the RAM model and Big Oh analysis provide **too coarse a set of tools** to make that type of distinction. When faced with algorithms of the same asymptotic complexity, implementation details and system quirks such as cache performance and memory size often prove to be the decisive factor.*
>
> *What we can say is that experiments show that when quicksort is implemented well, it is typically **two to three times faster** than mergesort or heapsort. The primary reason is that **the operations in the innermost loop are simpler**.*

---

6. The `Ω(n lg n)` Lower Bound

The comparison-sort model [CLRS §8.1, p.205]

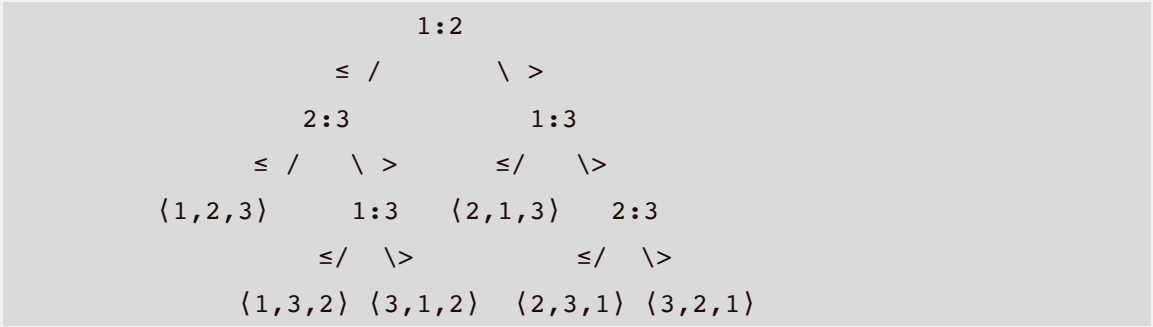
A **comparison sort** gains order information **only** through comparisons $a_i < a_j$, $a_i \le a_j$, $a_i = a_j$, $a_i \ge a_j$, $a_i > a_j$. It may not inspect the values or gain order information any other way.

Simplifications, all WLOG: assume distinct elements (a lower bound for distinct inputs applies when duplicates are allowed); then equality tests are useless; and the four remaining comparison forms are informationally equivalent, so assume all comparisons are $a_i \le a_j$.

The decision-tree model

A **decision tree** is a full binary tree representing the comparisons a particular algorithm performs on inputs of a given size. Internal nodes are labelled $i{:}j$ (compare a_i with a_j); leaves are labelled with a permutation $\langle \pi(1), \pi(2), \ldots, \pi(n) \rangle$.

Executing the algorithm = tracing a root-to-leaf path. Left = $a_i \le a_j$, right = $a_i > a_j$.



(insertion sort on three elements — CLRS Fig. 8.1 / Skiena Fig. 4.9)

The key requirement: because a correct sorting algorithm must be able to produce **every** permutation of its input, **each of the $n!$ permutations must appear as a reachable leaf**.

Theorem 8.1

> ***Any comparison sort algorithm requires $\Omega(n \lg n)$ comparisons in the worst case.***

Proof. The worst-case comparison count is the **height** of the decision tree. Let the tree have height h and l reachable leaves. Every one of the $n!$ permutations appears as a leaf, so $n! \le l$. A binary tree of height h has at most 2^h leaves, so

```
n! ≤ l ≤ 2^h
```

Taking logarithms (`lg` is monotonically increasing):

```
h ≥ lg(n!) = Ω(n lg n)          [by Stirling, CLRS eq. 3.28]
```

∎

Corollary 8.2. Heapsort and merge sort are **asymptotically optimal** comparison sorts.

Skiena's version of the same argument [§4.7.1, p.137]

> *Any sorting algorithm must behave differently during execution on each of the `n!` possible permutations of `n` keys.* ***If an algorithm did exactly the same thing with two different input permutations, there is no way that both of them could correctly come out sorted.***

That sentence is the cleanest statement of the intuition: **the algorithm's execution trace must be an injective function of the input permutation.** A binary decision tree of height `h` has only `2^h` distinct traces available.

Why this bound matters beyond sorting

> *This lower bound is important for several reasons. First, the idea can be extended to give lower bounds for* ***many applications of sorting****, including element uniqueness, finding the mode, and constructing convex hulls.* ***Sorting has one of the few non-trivial lower bounds among algorithmic problems.***

And the correct scoping of the model:

> ***Hashing-based algorithms do not perform such element comparisons, putting them outside the scope of this lower bound.*** *But hashing-based algorithms can get unlucky, and with worst-case luck the running time of any randomized algorithm for one of these problems will be `Ω(n log n)`.*

Corollaries worth knowing

- **No comparison sort is linear on even half the `n!` inputs** [Ex. 8.1-3]; nor on a `1/n` fraction, nor a `1/2ⁿ` fraction.

- **$\Omega(n \lg n)$ holds on average too**, not just worst case [CLRS Problem 8-1], via an external-path-length argument.
- **Randomization doesn't help** [Problem 8-1(f)]: for any randomized comparison sort there is a deterministic one making no more expected comparisons.
- The bound instantly rules out claims like "$O(1)$ insert, maximum, and extract-max" for priority queues [Skiena Ex. 4-40] — that would sort in $O(n)$.

---

7. The Linear-Time Sorts

All three escape $\Omega(n \lg n)$ by **leaving the comparison model** and using the keys' actual values. Each pays with an assumption.

Algorithm: Counting Sort

Assumption: each input is an **integer in $[0, k]$**.

Core intuition [CLRS §8.2, p.208]: for each x, determine how many elements are $\leq x$; that number *is* x's output position. *"If 17 elements are less than or equal to x, then x belongs in output position 17."* Adjust for duplicates by decrementing.

```
COUNTING-SORT(A, n, k)
 1  let B[1..n] and C[0..k] be new arrays
 2  for i = 0 to k:  C[i] = 0
 4  for j = 1 to n:  C[A[j]] = C[A[j]] + 1     // C[i] = count of
value i
 7  for i = 1 to k:  C[i] = C[i] + C[i-1]      // C[i] = count of
values ≤ i
11  for j = n downto 1                          // REVERSE order ->
stability
12      B[C[A[j]]] = A[j]
13      C[A[j]] = C[A[j]] - 1
14  return B
```

→ **C++ implementation:** A7 COUNTING-SORT

Complexity. $\Theta(k) + \Theta(n) + \Theta(k) + \Theta(n) = \Theta(k + n)$. Linear when $k = O(n)$.

Stability, and why line 11 counts down. Iterating `j` from `n` downto `1` and decrementing `C[A[j]]` places the *last* occurrence of a value at the *last* available slot for that value — preserving input order. **Iterating forward instead still sorts correctly but is not stable** [Ex. 8.2-3].

> *Counting sort's stability is important for another reason: counting sort is often used as a subroutine in **radix sort**. In order for radix sort to work correctly, counting sort must be stable.*

Why it beats the lower bound: *"no comparisons between input elements occur anywhere in the code. Instead, counting sort uses the actual values of the elements to index into an array."*

C++ Implementation

```cpp
#include <vector>

// Stable counting sort of values in [0, k]. Theta(n + k) time,
Theta(n + k) space.
vector<int> countingSort(const vector<int>& a, int k) {
    vector<int> count(k + 1, 0), out(a.size());
    for (int x : a) ++count[x];
    for (int i = 1; i <= k; ++i) count[i] += count[i - 1];   //
prefix sums
    // Backwards for stability: the last equal element takes the
last slot.
    for (int j = static_cast<int>(a.size()) - 1; j >= 0; --j)
        out[--count[a[j]]] = a[j];
    return out;
}
```

(Note the `--count[...]` idiom gives 0-based output positions directly.)

Bonus application [Ex. 8.2-6]: after `Θ(n + k)` preprocessing, the prefix-sum array answers "**how many of the `n` integers fall in `[a, b]`?**" in `O(1)` — as `C[b] - C[a-1]`. This is the 1-D prefix-sum trick in disguise.

Recognition pattern. Keys are small integers with a known bound, `k = O(n)`, and you need stability. Also: as the digit sort inside radix sort; as a `Θ(n)` bucket-counting step in many linear-time algorithms.

Algorithm: Radix Sort

Assumption: keys are `d`-digit numbers, each digit in `[0, k−1]`.

The counterintuitive part [CLRS §8.3, p.211]. Sorting on the **most** significant digit first requires setting aside 9 of 10 bins and recursing — generating a swarm of intermediate piles. Radix sort instead sorts on the **least significant digit first**, recombining the whole deck after each pass.

> *Remarkably, at that point the cards are fully sorted on the `d`-digit number. Thus,* ***only `d` passes*** *through the deck are required.*

```
RADIX-SORT(A, n, d)
1  for i = 1 to d
2      use a STABLE sort to sort array A[1..n] on digit i
```

→ **C++ implementation:** A8 RADIX-SORT

Stability is load-bearing. *"In order for radix sort to work correctly, the digit sorts must be stable."* Without stability, pass `i` destroys the ordering established by passes `1..i−1`.

Lemma 8.3. With a `Θ(n+k)` stable sort, radix sort is **`Θ(d(n + k))`**. Linear when `d` is constant and `k = O(n)`.

Lemma 8.4 — choosing the digit width. For `n` `b`-bit numbers and any `r ≤ b`, treat each key as `d = ⌈b/r⌉` digits of `r` bits, so `k = 2^r − 1`:

```
Θ((b/r)(n + 2^r))
```

The optimal `r`:

- If `b < ⌊lg n⌋`: choose `r = b`, giving `Θ(n)` — asymptotically optimal.
- If `b ≥ ⌊lg n⌋`: choose **`r = ⌊lg n⌋`**, giving `Θ(bn / lg n)`. Larger `r` makes `2^r` blow up faster than `r` helps; smaller `r` increases `b/r` while `n + 2^r` stays `Θ(n)`.

(CLRS's worked example: a 32-bit word as four 8-bit digits — `b = 32, r = 8, k = 255, d = 4`.)

Is radix sort better than quicksort? CLRS's honest answer:

> *If `b = O(lg n)`, as is often the case, and `r ≈ lg n`, then radix sort's running time is `Θ(n)`, which **appears** to be better than quicksort's `Θ(n lg n)`. **The constant factors hidden in the `Θ`-notation differ, however.** Although radix sort may make fewer passes, **each pass may take significantly longer.** ... **quicksort often uses hardware caches more effectively than radix sort** ... Moreover, radix sort with counting sort **does not sort in place**. Thus, when primary memory storage is at a premium, an in-place algorithm such as quicksort could be the better choice.*

C++ Implementation

```
#include <vector>
#include <cstdint>

// LSD radix sort of 32-bit unsigned ints, 8 bits per pass -> 4
passes.
// Theta(4(n + 256)) = Theta(n). Stable. Requires Theta(n) extra
space.
void radixSort(vector<uint32_t>& a) {
    const size_t n = a.size();
    if (n < 2) return;
    vector<uint32_t> buf(n);
    vector<size_t> count(256);

    for (int shift = 0; shift < 32; shift += 8) {
        fill(count.begin(), count.end(), 0);
        for (uint32_t x : a) ++count[(x >> shift) & 0xFFu];
        // Skip this pass entirely if every key shares the same
digit.
        if (count[(a[0] >> shift) & 0xFFu] == n) continue;
        size_t sum = 0;
        for (size_t d = 0; d < 256; ++d) { const size_t c =
count[d]; count[d] = sum; sum += c; }
        for (uint32_t x : a) buf[count[(x >> shift) & 0xFFu]++] =
x;   // stable
        a.swap(buf);
    }
}

// For SIGNED 32-bit ints: flip the sign bit to map int32 order
onto uint32 order,
// sort, then flip back.
void radixSortSigned(vector<int32_t>& a) {
```

```
    vector<uint32_t> u(a.size());
    for (size_t i = 0; i < a.size(); ++i)
        u[i] = static_cast<uint32_t>(a[i]) ^ 0x80000000u;
    radixSort(u);
    for (size_t i = 0; i < a.size(); ++i)
        a[i] = static_cast<int32_t>(u[i] ^ 0x80000000u);
}
```

Implementation notes.

- **Sign handling:** XOR the top bit maps `INT_MIN…INT_MAX` monotonically onto `0…UINT_MAX`. For IEEE floats the trick is different (flip all bits if negative, else flip only the sign bit).
- **Skipping uniform passes** is a large win on real data where high bytes are often constant.
- **Reduce to `d+1` passes** [Ex. 8.3-4]: counting sort makes two passes over the data per digit; you can compute all `d` digit histograms in a single initial pass, then do `d` distribution passes.
- **`n` integers in `[0, n³−1]` sort in `O(n)`** [Ex. 8.3-5]: view each as a 3-digit number in base `n`; `d = 3`, `k = n`, so `Θ(3(n+n)) = Θ(n)`.

Recognition pattern. Fixed-width integer or string keys, large `n`, memory to spare, no comparator needed. Also: sorting records by multiple fields — sort by the least significant field first with a stable sort.

Algorithm: Bucket Sort

Assumption: input is drawn **uniformly at random from `[0, 1)`**.

```
BUCKET-SORT(A, n)
1  let B[0..n−1] be a new array of empty lists
4  for i = 1 to n:  insert A[i] into list B[⌊n·A[i]⌋]
6  for i = 0 to n−1:  sort list B[i] with insertion sort
8  concatenate B[0], B[1], …, B[n−1] in order
```

→ **C++ implementation:** A9 BUCKET-SORT

Correctness. For `A[i] ≤ A[j]`, `⌊n·A[i]⌋ ≤ ⌊n·A[j]⌋`, so `A[i]` lands in the same or a lower-indexed bucket. Same bucket → line 6 orders them; different buckets → line 8 does.

Average-case analysis [CLRS §8.4, p.217].

```
T(n) = θ(n) + Σ_{i=0}^{n−1} O(nᵢ²)
```

where n_i is the number of elements in bucket i. Take expectations:

```
E[T(n)] = θ(n) + Σ E[O(nᵢ²)] = θ(n) + Σ O(E[nᵢ²])
```

Each n_i is **binomial** with n trials and $p = 1/n$, so $E[n_i] = 1$ and $Var[n_i] = np(1-p) = 1 - 1/n$. Then

```
E[nᵢ²] = Var[nᵢ] + E²[nᵢ] = (1 − 1/n) + 1 = 2 − 1/n
```

giving $E[T(n)] = \theta(n) + n \cdot O(2 - 1/n) = \theta(n)$. ✓

The condition is weaker than uniformity:

> *Even if the input is not drawn from a uniform distribution, bucket sort may still run in linear time.* ***As long as the sum of the squares of the bucket sizes is linear in the total number of elements****, bucket sort runs in linear time.*

Worst case is $\theta(n^2)$ — all elements into one bucket. Fix: sort each bucket with merge sort instead of insertion sort → $O(n \lg n)$ worst case, still $\theta(n)$ average [Ex. 8.4-2].

The Shifflett problem — when bucketing goes wrong

[Skiena §4.7, p.136] The best cautionary tale in either book.

> *Bucketing is a very effective idea whenever we are confident that the distribution of data will be roughly uniform. It is the idea that underlies hash tables, kd-trees, and a variety of other practical data structures.* ***The downside is that the performance can be terrible when the data distribution is not what we expected.*** *Although data structures such as balanced binary trees offer guaranteed worst-case behavior for* ***any*** *input distribution, no such promise exists for heuristic data structures on unexpected input distributions.*

> *Consider Americans with the uncommon last name of* ***Shifflett****. When last I looked, the Manhattan telephone directory (over one million names) contained exactly* ***five*** *Shiffletts. So how many Shiffletts should there be in a small city of 50,000 people? [The Charlottesville, Virginia telephone book has]* ***two and a half pages*** *of Shiffletts. …*

> *refining buckets from `S` to `Sh` to `Shi` to `Shif` to … to `Shifflett` results in **no significant partitioning**.*

The engineering lesson: every distribution-dependent structure — hash tables, bucket sort, kd-trees, sharding by key prefix — has a Shifflett. Real data has clusters that uniform-distribution analysis does not predict.

Choosing among the linear sorts

	ASSUMPTION	TIME	SPACE	STABLE	BREAKS WHEN
Counting	integers in `[0, k]`	`Θ(n + k)`	`Θ(n + k)`	✓	`k ≫ n`
Radix	`d`-digit, `k` values/digit	`Θ(d(n + k))`	`Θ(n + k)`	✓	many digits; cache-unfriendly
Bucket	uniform over `[0,1)`	`Θ(n)` avg, `Θ(n²)` worst	`Θ(n)`	✓	clustered data (**Shifflett**)

---

8. Order Statistics and Selection

The problem [CLRS Ch. 9, p.227]

> ***Input:*** *A set `A` of `n` distinct numbers and an integer `i`, `1 ≤ i ≤ n`.*
> ***Output:*** *The element `x ∈ A` that is larger than exactly `i − 1` other elements of `A`.*

The `i`-th **order statistic** is the `i`-th smallest. Minimum = 1st; maximum = `n`-th. The **median** is at `i = ⌊(n+1)/2⌋` (lower) and `⌈(n+1)/2⌉` (upper); CLRS says "the median" for the **lower** median.

Sorting solves it in `Θ(n lg n)`. **We can do `Θ(n)`.**

Minimum and maximum [CLRS §9.1]

Minimum alone: exactly `n − 1` comparisons, and that is optimal.

> *Think of any algorithm that determines the minimum as a **tournament** among the elements. Each comparison is a match in which the smaller element wins. Since **every element except the winner must lose at least one match**, `n − 1` comparisons are necessary.*

Both simultaneously: `3⌊n/2⌋` comparisons, not `2n − 2`.

The trick: **process elements in pairs.** Compare the pair to each other (1 comparison), then the smaller against the current min and the larger against the current max (2 more). **3 comparisons per 2 elements** instead of 4.

Initialization: if `n` is odd, set both min and max to the first element. If `n` is even, one comparison on the first two sets both. Either way the total is at most `3⌊n/2⌋`.

```
#include <vector>
#include <utility>
#include <algorithm>

// Both extremes in at most 3*floor(n/2) comparisons.
template <typename T>
pair<T,T> minMax(const vector<T>& v) {
    const int n = static_cast<int>(v.size());
    T mn, mx;
    int i;
    if (n % 2 == 1) { mn = mx = v[0]; i = 1; }
    else            { if (v[0] < v[1]) { mn = v[0]; mx = v[1]; }
                      else             { mn = v[1]; mx = v[0]; }
                      i = 2; }
    for (; i + 1 < n; i += 2) {              // 3 comparisons per
pair
        T lo = v[i], hi = v[i + 1];
        if (hi < lo) swap(lo, hi);
        if (lo < mn) mn = lo;
        if (mx < hi) mx = hi;
    }
    return {mn, mx};
}
```

Related exercises worth having thought about: second smallest in `n + ⌈lg n⌉ − 2` comparisons [Ex. 9.1-1] — run a tournament, then the second smallest must have lost directly to the winner, so search only the `⌈lg n⌉` elements the winner beat. And the classic **25 horses, 5 per race** puzzle: 6 races for the fastest, 7 for the fastest three [Ex.

9.1-3].

Algorithm: RANDOMIZED-SELECT (Quickselect)

Core intuition. Quicksort, but **recurse into only one side.**

```
RANDOMIZED-SELECT(A, p, r, i)
1  if p == r:  return A[p]
3  q = RANDOMIZED-PARTITION(A, p, r)
4  k = q - p + 1                    // rank of the pivot within
A[p..r]
5  if i == k:      return A[q]      // the pivot IS the answer
7  elseif i < k:   return RANDOMIZED-SELECT(A, p, q - 1, i)
9  else:           return RANDOMIZED-SELECT(A, q + 1, r, i - k)
```

→ **C++ implementation:** A10 RANDOMIZED-SELECT

Note `i - k` on the last line: you already know `k` values below the target, so you want the `(i-k)`-th smallest of the high side.

Worst case $\Theta(n^2)$ — always partition around the largest remaining element. Same recurrence as quicksort's worst case.

The intuition for linear expected time. Suppose the pivot lands in the **middle half** (2nd or 3rd quartile). Then whichever side we discard contains at least a quartile, so **at least** $1/4$ **of the elements leave play**:

```
T(n) = T(3n/4) + Θ(n)   →   master case 3   →   Θ(n)
```

The pivot lands in the middle half with probability ≈ $1/2$, so by the geometric distribution we expect **2 trials per success** — at most doubling the number of partitions, and each extra one is cheaper than the last.

The rigorous argument [CLRS §9.2, Theorem 9.2]

Setup. $A^{(j)}$ = set of elements still in play after j partitionings. Call the j-th partitioning **helpful** if $|A^{(j)}| \le (3/4)|A^{(j-1)}|$.

Lemma 9.1. A partitioning is helpful with probability at least `1/2`.
Proof sketch. Define the middle half of an `n`-element subarray as all but the smallest `⌈n/4⌉−1` and largest `⌈n/4⌉−1`. If the pivot falls there, at least `⌈n/4⌉` elements leave play, so at most `n − ⌈n/4⌉ = ⌊3n/4⌋ ≤ 3n/4` remain — helpful. The probability of *missing* the middle half is at most `2(⌈n/4⌉−1)/n ≤ (n/2)/n = 1/2`. ∎

Theorem 9.2. Group the partitionings into **generations**: generation `k` runs from the `k`-th helpful partitioning up to just before the `(k+1)`-st. Let `n_k = |A^{(h_k)}|`, so `n_k ≤ (3/4)^k n₀`. Let `X_k` be the number of partitionings in generation `k`; since each is helpful with probability ⩾ `1/2`, the geometric distribution gives `E[X_k] ≤ 2`.

The `j`-th partitioning makes fewer than `|A^{(j−1)}|` comparisons, so:

```
total comparisons < Σ_k Σ_{j in gen k} |A^{(h_k)}|  =  Σ_k X_k · n_k  ≤  Σ_k X_k (3/4)^k n₀
```

Taking expectations and using linearity:

```
E[…] = n₀ Σ_k (3/4)^k E[X_k]  ≤  2n₀ Σ_{k=0}^{∞} (3/4)^k  =  2n₀ · 4  =  8n₀
```

So `E[comparisons] = O(n)`; the first partition examines all `n`, giving `Ω(n)`. Hence **`Θ(n)` expected**. ∎

Note the shape of this proof — it is the same shape as the amortized arguments in M09: a geometrically shrinking sequence whose sum is a constant multiple of the first term.

Algorithm: SELECT (median of medians) — worst-case linear

> *This algorithm achieves linear time in the worst case, but it is **not nearly as practical** as `RANDOMIZED-SELECT`. It is mostly of theoretical interest. [CLRS §9.3, p.236]*

Core intuition. Guarantee a good pivot by **computing one recursively**: the median of the group medians.

The algorithm (CLRS 4e's version, which arranges groups by stride so no extra array is needed):

1. **While** `(r − p + 1) mod 5 ≠ 0`: move the minimum of `A[p..r]` to `A[p]`; if `i == 1` return it; otherwise `p++`, `i--`. *(Runs 0–4 times, making the size divisible by*

5.)

2. `g = (r − p + 1)/5` groups of 5. Group `j` is `⟨A[j], A[j+g], A[j+2g], A[j+3g], A[j+4g]⟩`. Sort each group in place. **All group medians now sit in `A[p+2g .. p+3g−1]` — the middle fifth.**
3. `x = SELECT(A, p+2g, p+3g−1, ⌈g/2⌉)` — the median of the medians, found **recursively**.
4. `q = PARTITION-AROUND(A, p, r, x)`; then proceed exactly as `RANDOMIZED-SELECT` lines 4–9.

Why the pivot is good. At least half the `g` group medians are `≤ x`, i.e. `⌈g/2⌉` groups. In each such group, 3 of the 5 elements (the median and the two below it) are `≤ x`. So **at least `3⌈g/2⌉ ≈ 3n/10` elements are `≤ x`**, and symmetrically `≈ 3n/10` are `≥ x`. Hence **each side of the partition holds at most about `7n/10` elements.**

The recurrence:

```
T(n) ≤ T(n/5) + T(7n/10) + Θ(n)
```

`n/5` for finding the median of medians, `7n/10` for the recursive select. Since `1/5 + 7/10 = 9/10 < 1`, this solves to **`Θ(n)`** — by substitution, or by Akra–Bazzi as worked in M03 §7.

Why groups of 5? With groups of 3 the recurrence becomes `T(n/3) + T(2n/3) + Θ(n) = Θ(n log n)` — the fractions sum to exactly 1 and it fails. 5 is the smallest odd group size that works.

C++ Implementation

```cpp
#include <vector>
#include <algorithm>
#include <random>
#include <utility>

namespace detail {

// 3-way partition around an explicit pivot VALUE. Returns [lt, gt] with
// v[lo..lt-1] < pivot, v[lt..gt] == pivot, v[gt+1..hi] > pivot.
template <typename T>
pair<int,int> partitionAround(vector<T>& v, int lo, int hi, const T& pivot) {
```

```
    int lt = lo, i = lo, gt = hi;
    while (i <= gt) {
        if      (v[i] < pivot) swap(v[lt++], v[i++]);
        else if (pivot < v[i]) swap(v[i], v[gt--]);
        else                   ++i;
    }
    return {lt, gt};
}

}  // namespace detail

// Quickselect: v[k] becomes the k-th smallest (0-indexed); the
array is
// partially reordered. O(n) expected, O(n^2) worst case.
template <typename T>
T quickSelect(vector<T> v, int k) {
    mt19937 rng(random_device{}());
    int lo = 0, hi = static_cast<int>(v.size()) - 1;
    while (lo < hi) {
        uniform_int_distribution<int> pick(lo, hi);
        const T pivot = v[pick(rng)];
        auto [lt, gt] = detail::partitionAround(v, lo, hi, pivot);
        if      (k < lt) hi = lt - 1;
        else if (k > gt) lo = gt + 1;
        else             return v[k];      // k lands inside the
equal block
    }
    return v[lo];
}

// Median-of-medians: O(n) WORST case. Slower in practice than
quickSelect.
template <typename T>
T selectWorstCaseLinear(vector<T> v, int k) {
    vector<T> cur = move(v);
    while (true) {
        if (cur.size() <= 5) {
            sort(cur.begin(), cur.end());
            return cur[k];
        }
        // Median of each group of 5.
        vector<T> medians;
```

```
        medians.reserve((cur.size() + 4) / 5);
        for (size_t i = 0; i < cur.size(); i += 5) {
            const size_t j = min(i + 5, cur.size());
            sort(cur.begin() + i, cur.begin() + j);
            medians.push_back(cur[i + (j - i - 1) / 2]);
        }
        const T pivot = selectWorstCaseLinear(medians,
                                              static_cast<int>
(medians.size() - 1) / 2);
        auto [lt, gt] = detail::partitionAround(cur, 0,
static_cast<int>(cur.size()) - 1, pivot);
        if (k < lt) { cur.resize(lt); }
        else if (k > gt) {
            cur.erase(cur.begin(), cur.begin() + (gt + 1));
            k -= (gt + 1);
        } else {
            return cur[k];
        }
    }
}
```

Implementation notes

- **The 3-way partition matters even more here than in quicksort.** With many duplicates, a 2-way quickselect can fail to shrink the range at all.
- `quickSelect` **takes** `v` **by value** — selection permutes the array. Take a reference if you want the side effect (that's what `std::nth_element` does).
- **In practice use** `std::nth_element` — it is introselect (quickselect with a median-of-medians fallback after too many bad partitions), giving `O(n)` expected with a worst-case guarantee.
- **Median of medians is genuinely slow.** Its constant factor is large enough that quickselect beats it on essentially all real inputs. Know it for the theory and the recurrence.

Common bugs

- Forgetting `i - k` when recursing into the high side.
- Recursing on `[lo, q]` rather than `[lo, q-1]` → infinite loop.
- Two-way partitioning with duplicates → no progress.

- Groups of 3 in median-of-medians → `Θ(n log n)`.

Recognition pattern

- "**`k`-th smallest/largest**" with `k` fixed → quickselect, `O(n)`.
- "**Top `k`**" with `k ≪ n` → a size-`k` heap, `O(n log k)`, or `std::partial_sort`.
- "**Median**" → quickselect at `n/2`.
- "**Median of two sorted arrays**" → binary search on the partition point, `O(log(min(m,n)))` — a different technique (M03).
- **`k` smallest elements** (unordered) → `std::nth_element` then take the prefix, `O(n)`.

---

9. War Story: Skiena for the Defense — external sorting

[Skiena §4.8, p.138]

Called as an expert witness in a case about high-performance sorting programs, Skiena expected to learn which in-place sort is fastest in practice.

> *The answer was quite humbling.* ***Nobody cared about in-place sorting.*** *The name of the game was sorting huge files, much bigger than can fit in main memory.* ***All the important action was in getting the data on and off a disk.*** *Cute algorithms for doing internal (in-memory) sorting were not the bottleneck.*

> *Recall that disks have relatively long seek times… Once the head is in the right place, the data moves relatively quickly, and it costs about the same to read a large data block as it does to read a single byte. Thus,* ***the goal is minimizing the number of blocks read/written****, and coordinating these operations so the sorting algorithm is never waiting to get the data it needs.*

The winning algorithm: multiway merge sort.

> *You build a heap with members of the top block from each of `k` sorted lists. By repeatedly plucking the top element off this heap, you build a sorted list merging these `k` lists. Because this heap is sitting in main memory, these operations are fast. When you have a large enough sorted run, you write it to disk and free up memory for more data. When you get close to emptying out the elements from the top block of one of the `k` sorted lists, load the next block.*

The Minutesort benchmark (sort as much data as possible in one minute): at Skiena's writing, Tencent Sort had sorted **55 terabytes in under a minute** on a 512-node cluster (20 cores, 512 GB RAM each). Current records at `sortbenchmark.org`.

Benchmarking is hard: *"Is it fair to compare a commercial program designed to handle general files with a stripped-down code optimized for integers?"* And a widely used trick: **strip off a short prefix of the key and sort on that first**, to avoid moving all those extra bytes around.

The technical lesson:

> *It is important to worry about external memory performance whenever you combine **very large datasets with low-complexity algorithms** (say linear or `Θ(n log n)`). **Constant factors of even 5 or 10 can make a big difference here between what is feasible and what is hopeless.** Of course, quadratic-time algorithms are doomed to fail on large datasets regardless of data access times.*

(And the non-technical lesson, which Skiena rates first: "do everything you can to avoid being involved in a lawsuit either as a plaintiff or defendant.")

---

10. The comparison table

ALGORITHM	BEST	AVERAGE / EXPECTED	WORST	SPACE	STABLE	IN PLACE	NOTES
Insertion sort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	✓	✓	$\Theta(n + \text{inversions})$; online; base case of hybrid sorts
Selection sort	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	✗	✓	Identical on all $n!$ inputs; minimizes **writes** ($n-1$ swaps)
Merge sort	$\Theta(n \lg n)$	$\Theta(n \lg n)$	$\Theta(n \lg n)$	$\Theta(n)$	✓	✗	Best for linked lists and external sorting
Heapsort	$\Theta(n \lg n)$	$O(n \lg n)$	$O(n \lg n)$	$O(1)$	✗	✓	Cache-hostile; introsort's fallback
Quicksort	$\Theta(n \lg n)$	$\Theta(n \lg n)$ exp.	$\Theta(n^2)$	$O(\lg n)$ stack	✗	✓	Fastest constants; needs 3-way for duplicates
Counting sort	$\Theta(n+k)$	$\Theta(n+k)$	$\Theta(n+k)$	$\Theta(n+k)$	✓	✗	Integers in $[0,k]$
Radix sort	$\Theta(d(n+k))$	$\Theta(d(n+k))$	$\Theta(d(n+k))$	$\Theta(n+k)$	✓	✗	d-digit keys; stable digit sort required
Bucket sort	$\Theta(n)$	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n)$	✓	✗	Uniform $[0,1)$; Shifflett risk
Quickselect	$\Theta(n)$	$\Theta(n)$ exp.	$\Theta(n^2)$	$O(1)$	—	✓	i-th order statistic
Median of medians	$\Theta(n)$	$\Theta(n)$	**$\Theta(n)$**	$O(\lg n)$	—	✓	Theoretical; large constant

---

Chapter in One Page

CONCEPT	THE ONE-LINE VERSION
Why sort	It makes search, closest pair, uniqueness, mode, selection, and convex hull easy. Try it first.
Sorting vs hashing	Hashing answers **equality**; sorting answers **order**.
Set intersection	**Sort the smaller set**: `O((n+m) log m)`.
Pragmatics	Ask: direction? key or record? equal keys? non-numeric collation?
Stability	Equal elements keep input order. Achievable for any sort by appending the index as a tiebreak.
Heap property	Max-heap: `A[parent(i)] ≥ A[i]`. Nearly complete tree in an array; height `⌊lg n⌋`.
Heap indices	1-indexed: `2i`, `2i+1`, `⌊i/2⌋`. 0-indexed: `2i+1`, `2i+2`, `(i−1)/2`.
You cannot search a heap	Nothing is known about the relative order of the `n/2` leaves.
`MAX-HEAPIFY`	`O(h)`; `T(n) ≤ T(2n/3) + Θ(1) = O(lg n)`.
`BUILD-MAX-HEAP`	**`Θ(n)`**, via `Σ h/2^h ≤ 2` and `≤ ⌈n/2^{h+1}⌉` nodes of height `h`.
Build ≠ repeated insert	Bottom-up is `Θ(n)`; `n` inserts is `Θ(n log n)`.
Heapsort	*Selection sort with the right data structure.* `O(n lg n)`, in place, unstable.
Priority queue	`INSERT`, `MAXIMUM`, `EXTRACT-MAX`, `INCREASE-KEY`, all `O(lg n)`.
Handles	`DECREASE-KEY` needs object→index mapping; `std::priority_queue` has none → lazy deletion.
X + Y pattern	PQ as a lazy frontier over `(i,j)` pairs — the airline-fare war story.
Quicksort partition	Four regions; loop invariant `≤x | >x | unknown | pivot`.
Split proportionality	**Any** constant-ratio split (9:1, 99:1) gives `O(n lg n)`. Only the constant changes.
Bad split absorbed	A bad split followed by a good one costs the same as a single good split.
Good-enough pivot	Half of all pivots land in `[n/4, 3n/4]`; average tree height ≈ `2 ln n ≈ 1.386 lg n`.
Lemma 7.2	`zᵢ` and `z_j` are compared **iff** one of them is the first pivot chosen from `Z_{ij}`.
Lemma 7.3	`Pr{compared} = 2/(j − i + 1)`.

Theorem 7.4	$E[X] = \Sigma\,\Sigma\ 2/(j-i+1) = O(n \lg n)$ via the harmonic series.
Quicksort worst case	$\Theta(n^2)$, on the **already sorted** array (which insertion sort does in $O(n)$).
3-way partition	Mandatory with duplicates; otherwise all-equal input is $\Theta(n^2)$.
Stack depth	Recurse on the **smaller** side, loop on the larger → $\Theta(\lg n)$ stack.
Decision tree	Comparison sorts ⇒ binary tree with $\geqslant n!$ reachable leaves.
Theorem 8.1	$n! \leq 2^h \implies h \geq \lg(n!) = \Omega(n \lg n)$. Heapsort and merge sort are optimal.
Skiena's phrasing	The execution trace must differ for each of the $n!$ permutations.
Counting sort	$\Theta(n+k)$, stable; count → prefix-sum → place **backwards**.
Radix sort	$\Theta(d(n+k))$, **LSD first**, requires a **stable** digit sort. Optimal $r = \lfloor \lg n \rfloor$.
Bucket sort	$\Theta(n)$ average on uniform $[0,1)$; needs only $\Sigma n_i^2 = O(n)$.
Shifflett	Real data clusters; distribution-dependent structures have no worst-case promise.
Min alone	Exactly $n-1$ comparisons — a tournament: every non-winner loses once.
Min **and** max	$3\lfloor n/2 \rfloor$ comparisons — process in **pairs**.
Quickselect	Recurse into one side only. $\Theta(n)$ expected, $\Theta(n^2)$ worst.
Helpful partition	Middle-half pivot ⇒ $\leq 3n/4$ survive; probability $\geq 1/2$; generations sum to $8n_0$.
Median of medians	Groups of 5 → $T(n) \leq T(n/5) + T(7n/10) + \Theta(n) = \Theta(n)$. Groups of 3 fail.
External sorting	Multiway merge with an in-memory heap; minimize **block** I/O, not comparisons.

Recognition Table

CLUE	TECHNIQUE
"Find duplicates / closest pair / mode"	sort first
Equality only, order irrelevant	hash, not sort
Two sets, one much smaller	sort the **small** one, binary search the big one
Need stability	merge sort / `std::stable_sort` / counting / radix
Need worst-case `n log n` **and** `O(1)` space	heapsort
General in-memory sort	quicksort / `std::sort`
Linked list	merge sort
Data doesn't fit in memory	external multiway merge sort
Keys are small integers, `k = O(n)`	counting sort
Fixed-width integer/string keys, huge `n`	radix sort
Uniformly distributed reals	bucket sort — but check for Shiffletts
"`k`-th smallest", `k` given	quickselect / `std::nth_element` — `O(n)`
"Top `k`", `k ≪ n`	size-`k` heap — `O(n log k)`
"Merge `k` sorted lists"	min-heap of `k` heads — `O(n log k)`
"`k` smallest pairs / sorted matrix"	PQ frontier over `(i,j)` — the X+Y pattern
"Running median of a stream"	two heaps (max-heap below, min-heap above)
"Count inversions"	merge sort with a counter
Must prove no algorithm can be faster	decision tree, `n!` leaves
Someone claims `O(1)` PQ operations	it would sort in `O(n)` — impossible
Data almost sorted	insertion sort (`Θ(n + inversions)`) or Timsort
Sort by several fields	stable sort on the **least** significant field first

---

Common Mistakes Recap

1. Claiming `BUILD-MAX-HEAP` is `O(n log n)`. It is `Θ(n)`.
2. Building a heap by `n` insertions and calling it linear. That is `Θ(n log n)`.
3. Trying to search a heap efficiently.
4. Mixing 0-indexed and 1-indexed heap child formulas.

5. Assuming heapsort or quicksort is stable.
6. Plain 2-way quicksort on data with many duplicates → $\Theta(n^2)$.
7. Not eliminating tail recursion → $\Theta(n)$ stack in the worst case.
8. Counting sort iterating **forward** in the placement loop, losing stability.
9. Radix sort with an unstable digit sort. Silently wrong.
10. MSD-first radix sort without appreciating the pile-tracking cost.
11. Bucket sort on clustered data (the Shifflett problem).
12. Forgetting `i - k` when quickselect recurses right.
13. Median-of-medians with groups of 3.
14. Claiming a comparison sort beats $\Omega(n \lg n)$ — or forgetting that counting/radix/bucket are simply *not comparison sorts*.
15. Optimizing comparisons when sorting data larger than memory, where **block I/O** dominates.

Self-Test

1. Name six problems that become easy after sorting. Which of them can hashing do faster, and which can it not do at all? *(§1)*
2. Two sets of sizes $m \ll n$: give the best sorting-based disjointness test and its complexity. *(§1)*
3. What four questions should you ask before choosing a sort? *(§2)*
4. Define stability. Which of the eight sorts here are stable? How do you stabilize any sort? *(§2)*
5. Give the heap property, the index formulas, and the height of an n-element heap. *(§3)*
6. Why can't you search a heap efficiently? *(§3)*
7. Prove `BUILD-MAX-HEAP` runs in $\Theta(n)$. Which two facts does the proof need? *(§3)*
8. Why does the build loop run from $\lfloor n/2 \rfloor$ **downto** 1? *(§3)*
9. Why is heapsort "selection sort with the right data structure"? *(§3)*
10. Why does `MAX-HEAP-INSERT` set the new key to $-\infty$ first? Why can't `MAX-HEAPIFY` replace the bubble-up loop? *(§4)*
11. Describe the airline-fare priority-queue algorithm. How do you avoid duplicates? *(§4)*
12. Determine in $O(k)$ whether the k-th smallest heap element is $\geq x$. Why is it

`O(k)`? *(§4)*

13. State quicksort's partition loop invariant and discharge maintenance. *(§5)*
14. Show that a 9-to-1 split at every level still gives `O(n lg n)`. What does the ratio affect? *(§5)*
15. State Lemma 7.2 exactly, and prove it in three cases. *(§5)*
16. Derive `E[X] = O(n lg n)` from Lemma 7.3. *(§5)*
17. What is quicksort's worst-case input, and what is ironic about it? *(§5)*
18. Prove `Ω(n lg n)` for comparison sorts. Where does `n!` enter, and where does `2^h`? *(§6)*
19. Why doesn't the bound apply to counting sort? To hashing-based algorithms? *(§6)*
20. Write counting sort. Why does the placement loop run backwards? *(§7)*
21. Why must radix sort's digit sort be stable, and why LSD first? *(§7)*
22. Given `n` `b`-bit numbers, what `r` minimizes `(b/r)(n + 2^r)`, and why? *(§7)*
23. Prove bucket sort's `θ(n)` average case. What is the actual sufficient condition? *(§7)*
24. What is the Shifflett problem and what does it teach? *(§7)*
25. Why does finding the minimum require exactly `n−1` comparisons? *(§8)*
26. Find min **and** max in `3⌊n/2⌋` comparisons. *(§8)*
27. Write quickselect. Why `i − k` on the right branch? *(§8)*
28. Define a "helpful" partition and sketch the $8n_0$ bound. *(§8)*
29. Give median-of-medians' recurrence and explain where `n/5` and `7n/10` come from. Why not groups of 3? *(§8)*
30. Why did external sorting turn out not to care about in-place algorithms? *(§9)*

Practice — where to drill this module

IDEA IN THIS MODULE	PROBLEM	WHY IT'S THE RIGHT DRILL
Write a sort yourself	912 · Sort an Array	`std::sort` is banned in spirit; submit heapsort, then merge sort, then quicksort — and watch the deterministic pivot TLE
Heap as a priority queue	215 · Kth Largest Element in an Array · 703 · Kth Largest Element in a Stream	215 wants quickselect *or* a size-`k` heap; 703 is the streaming version
Two heaps	295 · Find Median from Data Stream	max-heap + min-heap; the single most-asked heap design question
`MERGE` as the combine step	23 · Merge k Sorted Lists	heap-of-`k`-heads is `Θ(n lg k)`; pairwise merging is the D&C route
Counting frequencies then ordering	347 · Top K Frequent Elements	bucket sort by frequency gives `Θ(n)` — the linear-sort idea applied
`QUICKSELECT`	215 · Kth Largest Element in an Array	`RANDOMIZED-SELECT` verbatim; `A10` below has both recursive and iterative
Stability matters	937 · Reorder Data in Log Files	`stable_sort` vs `sort` gives different accepted/rejected answers
Counting while merging	493 · Reverse Pairs	the merge step does the work; see M03
Sorting by a custom order	179 · Largest Number	the comparator is the whole problem — and it must be a **strict weak ordering**

Beyond LeetCode. CSES Problem Set — *Sorting and Searching* is the best single set of drills for this module. Codeforces `sortings` tag · `data structures` tag.

The drill that matters here: for every sort you write, answer four questions without looking — *in place? stable? worst case? what input triggers it?* `A5` below shows what happens when you get the last one wrong.

C++ Toolkit for This Module

*Language material from Weiss, **Data Structures and Algorithm Analysis in C++**, 4th ed., ch. 1 and §6.9 / §7.2.2.*

1. Comparators, and the strict-weak-ordering contract

Weiss motivates function objects [§1.6.4, p.41] with precisely the sorting problem: a `Rectangle` has no natural `<`, so the ordering must be passed in. Three ways, all equivalent to the compiler:

```
struct ByLength {                                          // 1. a
function object (Weiss's form)
    bool operator()(const string& a, const string& b) const {
return a.size() < b.size(); }
};
bool byLengthFn(const string& a, const string& b) { return a.size()
< b.size(); }  // 2. free function

void comparatorDemo(vector<string>& v) {
    sort(v.begin(), v.end(), ByLength{});
    sort(v.begin(), v.end(), byLengthFn);
    sort(v.begin(), v.end(),                               // 3. a
lambda -- same thing, less typing
         [](const string& a, const string& b) { return a.size() <
b.size(); });
}
```

The contract `std::sort` requires is a *strict weak ordering*. Concretely: `cmp(a, a)` must be `false`. Writing `<=` instead of `<` violates it, and libstdc++'s introsort will then **run off the end of the array** — a real segfault, not a wrong answer, and only on inputs big enough to hit the quicksort path. This is the most expensive one-character bug in competitive C++.

2. `sort`, `stable_sort`, `partial_sort`, `nth_element`

FUNCTION	GUARANTEE	COST	USE WHEN
`sort`	not stable	`O(n lg n)` worst case (introsort)	the default
`stable_sort`	**stable**	`O(n lg² n)`, or `O(n lg n)` with memory	equal keys must keep input order
`partial_sort(b, m, e)`	first `m` sorted	`O(n lg m)`	"top `m`"
`nth_element(b, nth, e)`	`*nth` is correct; both sides partitioned	**`O(n)` expected**	**this is `RANDOMIZED-SELECT`**
`is_sorted`, `is_heap`	—	`O(n)`	assertions in tests

`nth_element` is the standard library's `RANDOMIZED-SELECT` (`A10`), and `priority_queue` is `MAX-HEAP` (`A4`). Knowing that the STL *is* this chapter is half the value of the chapter.

3. `priority_queue` is a **max**-heap by default

```
void pqDemo() {
    priority_queue<int> maxq;
   // top() is the LARGEST
    priority_queue<int, vector<int>, greater<int>> minq;
    // top() is the SMALLEST
    // The comparator is the THIRD template argument, so you must
spell out the
    // second (the container) even though you did not want to
change it.
    (void)maxq; (void)minq;
}
```

Getting this backwards is the second-most-common heap bug. And note `priority_queue` has **no `decrease-key`** — which is why M14 and M15 use the "push a new entry and skip stale pops" idiom instead.

4. 1-based arrays: why this module wastes index 0

CLRS's heap indexing depends on it:

```
inline int PARENT(int i) { return i / 2; }      // 1-based: clean
inline int LEFT(int i)   { return 2 * i; }
inline int RIGHT(int i)  { return 2 * i + 1; }
// 0-based equivalents are uglier and easier to get wrong:
//   parent = (i - 1) / 2,  left = 2i + 1,  right = 2i + 2
```

The appendix keeps 1-based indexing (`A[0]` unused) so every line matches CLRS. In production, use the 0-based forms — or better, use `priority_queue`.

5. `swap` is three moves, not three copies

Weiss [§1.5.5, p.29]: `std::swap` on a large type used to cost three full copies; since C++11 it is three **moves** — *"little more than a pointer change"* for a `vector` or `string`. That is why `HEAPSORT` and `PARTITION`, which are swap-heavy, are cheap even on heavy element types. It is also why `swap(a, a)` is harmless but not free: guard it with `if (i != j)` when the element type is expensive.

6. Templates and `Comparable` — the assumption goes in a comment

Weiss [§1.6.1, p.37]: *"it is customary to include, prior to any template, comments that explain what assumptions are made about the template argument(s)."* The appendix below is written for `int` to keep the pseudocode correspondence exact; the templated versions in the body state their `Comparable` requirement in a comment, as Weiss instructs.

7. Instrumentation: counting without changing the algorithm

Every appendix entry threads a `long long` counter through so its complexity claim can be measured:

```
long long partitionCompares = 0;   // a file-scope counter, reset
before each measurement
```

A global is the wrong design for production code (not thread-safe, invisible coupling) and the right one for a notes file, where the alternative — an extra parameter on every function — would obscure the correspondence with the pseudocode. **`long long`, never `int`:** at `n = 4000` the sorted-array quicksort below performs 7 998 000 comparisons, and $n^2/2$ at `n = 100 000` is 5×10^9.

Appendix — C++ for Every Pseudocode Block

Shared prelude — the random source (see M04) and CLRS's heap index functions:

```
mt19937& rng() { static mt19937 gen(20260903u); return gen; }   //
fixed seed: reproducible
int randomInt(int a, int b) { return uniform_int_distribution<int>
(a, b)(rng()); }

// 1-BASED heap indexing, exactly as CLRS. A[0] is unused
throughout this appendix.
inline int PARENT(int i) { return i / 2; }
inline int LEFT(int i)   { return 2 * i; }
inline int RIGHT(int i)  { return 2 * i + 1; }
```

A1 MAX-HEAPIFY

Pseudocode: §3, *"Algorithm: MAX-HEAPIFY"*.

```
long long heapifySteps = 0;      // instrumentation (toolkit 7)

// Precondition: the subtrees rooted at LEFT(i) and RIGHT(i) are
max-heaps, but
// A[i] may be smaller than a child. Postcondition: the subtree at
i is a max-heap.
//
// `heapSize` is passed explicitly rather than being A.size():
HEAPSORT shrinks
// the heap while leaving the sorted suffix in the same array, so
"how much of A
// is still a heap" is NOT a property of the vector.
void maxHeapify(vector<int>& A, int i, int heapSize) {
    int l = LEFT(i), r = RIGHT(i);
    int largest;
    // SHORT-CIRCUIT ORDER MATTERS: `l <= heapSize` must come
first, or A[l] is
    // read out of bounds when i is a leaf. && evaluates left to
right and stops.
    if (l <= heapSize && A[l] > A[i]) largest = l;
    else                              largest = i;
    if (r <= heapSize && A[r] > A[largest]) largest = r;
    if (largest != i) {
```

```
        ++heapifySteps;
        swap(A[i], A[largest]);
        maxHeapify(A, largest, heapSize);      // TAIL call -- see
the loop version
    }
}

// The recursive call above is in tail position, so this is the
mechanical
// conversion. Prefer it: the depth is only lg n, but writing the
loop costs
// nothing and removes any dependence on the compiler doing TCO
(M03 toolkit 2).
void maxHeapifyIterative(vector<int>& A, int i, int heapSize) {
    for (;;) {
        int l = LEFT(i), r = RIGHT(i), largest = i;
        if (l <= heapSize && A[l] > A[largest]) largest = l;
        if (r <= heapSize && A[r] > A[largest]) largest = r;
        if (largest == i) return;          // A[i] is already >=
both children: done
        swap(A[i], A[largest]);
        i = largest;                       // "recurse" by
reassigning the parameter
    }
}
```

Complexity. `T(n) ≤ T(2n/3) + Θ(1)` → master case 2 → **`O(lg n)`**. The `2n/3` is the worst case: when the bottom level is exactly half full, one subtree holds up to `2n/3` nodes. Equivalently, the cost is `O(h)` for a node at height `h`.

> *Verified: the recursive and iterative forms produce **identical arrays** on 400 heaps with a deliberately corrupted root.*

A2 BUILD-MAX-HEAP

Pseudocode: §3, "Algorithm: BUILD-MAX-HEAP".

```
void buildMaxHeap(vector<int>& A) {
    const int n = (int)A.size() - 1;
    // Start at floor(n/2), not at n: elements A[n/2+1 .. n] are
LEAVES, and a
```

```
    // leaf is already a one-element max-heap. Half the array needs
no work at all.
    // Go DOWNWARD so that when maxHeapify(i) runs, both children
of i are
    // already heaps -- that is its precondition.
    for (int i = n / 2; i >= 1; --i)
        maxHeapify(A, i, n);
}

bool isMaxHeap(const vector<int>& A, int heapSize) {
    for (int i = 2; i <= heapSize; ++i)
        if (A[PARENT(i)] < A[i]) return false;
    return true;
}
```

Complexity — `Θ(n)`, not `Θ(n lg n)`. The loose bound is "`n/2` calls × `O(lg n)` each". The tight one uses the fact that **most nodes are near the bottom, where `maxHeapify` is cheap**: there are at most `⌈n/2^{h+1}⌉` nodes of height `h`, and each costs `O(h)`, so

```
Σ_{h=0}^{⌊lg n⌋} ⌈n/2^{h+1}⌉ · O(h)  =  O( n · Σ_{h=0}^{∞} h/2^h )
=  O(n · 2)  =  O(n)
```

using `Σ h/2^h = 2` (the standard geometric-derivative sum from M02).

> *Verified — measured swaps, against both `n` and `n lg n`:*
>
> | *N* | *SWAPS* | *SWAPS / N* | *N LG N* |
> |---|---|---|---|
> | *1 000* | *720* | *0.720* | *9 966* |
> | *10 000* | *7 470* | *0.747* | *132 877* |
> | *100 000* | *74 581* | *0.746* | *1 660 964* |
> | *1 000 000* | ***744 192*** | ***0.744*** | *19 931 569* |
>
> *The ratio converges to a constant ≈ 0.744, which is exactly what `Θ(n)` means — and it is **27× below** `n lg n` at `n = 10⁶`. This is the measurement that makes the tight analysis believable.*

A3 HEAPSORT

Pseudocode: §3, "Algorithm: Heapsort".

```
void heapsort(vector<int>& A) {
    const int n = (int)A.size() - 1;
    buildMaxHeap(A);                    // 1  Theta(n)
    int heapSize = n;
    for (int i = n; i >= 2; --i) {      // 2  n-1 iterations
        swap(A[1], A[i]);               // 3  the max goes to its
FINAL position
        --heapSize;                     // 4  and leaves the heap
forever
        maxHeapify(A, 1, heapSize);     // 5  O(lg n) to restore
the root
    }
    // Note what the array looks like throughout: A[1..heapSize] is
a heap and
    // A[heapSize+1..n] is the sorted suffix, already in final
position. The two
    // regions share one array -- which is why heapsort is IN
PLACE.
}
```

Complexity. `Θ(n) + (n−1)·O(lg n) = Θ(n lg n)` — **worst, average and best case alike**. Space `Θ(1)` auxiliary. **Not stable.**

Why it is not the default sort in practice, despite the perfect worst case: every `maxHeapify` jumps between `i`, `2i` and `2i+1`, which are far apart in memory once the heap exceeds cache. Quicksort's partition is a **linear scan** and wins on cache behaviour by a factor of 2–3. `std::sort` uses introsort — quicksort, with a switch to heapsort when the recursion gets too deep — to get quicksort's speed *and* `O(n lg n)` worst case.

> *Verified: matches `std::sort` on 400 random arrays.*

A4 MAX-HEAP-INSERT and MAX-HEAP-INCREASE-KEY

Pseudocode: §3, "`INCREASE-KEY` and `INSERT`".

```
// A max-priority-queue on ints. Compare with std::priority_queue,
which is this
// class with the same algorithms and a nicer interface.
class MaxPriorityQueue {
public:
```

```
    int size() const { return (int)heap_.size() - 1; }   // -1 for
the unused slot 0
    bool empty() const { return size() == 0; }

    // MAX-HEAP-INSERT: append at -infinity, then increase the key
into place.
    // The two-step dance is not decoration -- it is how the
pseudocode reuses
    // INCREASE-KEY's sift-up loop instead of writing a second one.
    void insert(int key) {
        heap_.push_back(numeric_limits<int>::min());     // x.key =
-infinity
        increaseKey(size(), key);                        // then
raise it to `key`
    }

    // `.at(1)` not `[1]`: at() throws std::out_of_range on an
empty queue,
    // while operator[] is UNDEFINED BEHAVIOUR -- it would read
past the end and
    // return whatever was there. In a data structure whose whole
job is
    // bookkeeping, prefer the checked accessor at the boundary.
    int maximum() const { return heap_.at(1); }

    int extractMax() {
        int mx = heap_.at(1);
        heap_[1] = heap_.back();       // move the LAST leaf to the
root
        heap_.pop_back();
        if (size() >= 1) siftDown(1);  // and let it sink
        return mx;
    }

    // MAX-HEAP-INCREASE-KEY, the sift-UP loop.
    void increaseKey(int i, int key) {
        // The pseudocode says error "new key is smaller than
current key".
        // An exception is the C++ way to say that: it cannot be
ignored, unlike
        // a returned error code, and it carries a message.
```

```
        if (key < heap_[i]) throw invalid_argument("new key is
smaller than current key");
        heap_[i] = key;
        while (i > 1 && heap_[PARENT(i)] < heap_[i]) {   // i > 1
FIRST: no parent of the root
            swap(heap_[i], heap_[PARENT(i)]);
            i = PARENT(i);
        }
    }
private:
    // heap_[0] is a permanently unused sentinel so that
PARENT/LEFT/RIGHT are
    // the clean 1-based formulas. The `{0}` initialises the vector
with that
    // one dummy element.
    vector<int> heap_{0};

    void siftDown(int i) {
        const int n = size();
        for (;;) {
            int l = LEFT(i), r = RIGHT(i), largest = i;
            if (l <= n && heap_[l] > heap_[largest]) largest = l;
            if (r <= n && heap_[r] > heap_[largest]) largest = r;
            if (largest == i) return;
            swap(heap_[i], heap_[largest]);
            i = largest;
        }
    }
};
```

Complexity. `INSERT`, `INCREASE-KEY`, `EXTRACT-MAX`: **`O(lg n)`**. `MAXIMUM`: `O(1)`.

The line the pseudocode hides. Step 3 of `MAX-HEAP-INCREASE-KEY` says *"find the index `i` where object `x` occurs"* — that is `Θ(n)` unless you maintain an object→index map alongside the heap, updating it on **every swap**. CLRS 4e makes this explicit; most textbook implementations quietly ignore it. It is exactly why `std::priority_queue` offers no `decrease-key`, and why Dijkstra (M15) uses the lazy "push a duplicate, skip stale pops" idiom instead of maintaining that map.

> *Verified: against `std::priority_queue` over **20 000 random insert / peek / extract operations** — identical results and identical sizes throughout. `increaseKey` with a smaller key throws, as specified.*

A5 QUICKSORT and PARTITION

Pseudocode: §4, "Pseudocode".

```
long long partitionSwaps = 0, partitionCompares = 0;

// LOMUTO partition. Invariant, maintained for every j:
//     A[p   .. i  ] <= x        (the "low" side)
//     A[i+1 .. j-1] >  x        (the "high" side)
//     A[j   .. r-1] unexamined
//     A[r] == x                 (the pivot, parked at the end)
int partitionLomuto(vector<int>& A, int p, int r) {
    int x = A[r];                       // 1  pivot = LAST element
    int i = p - 1;                      // 2  low side is empty
    for (int j = p; j <= r - 1; ++j) {  // 3
        ++partitionCompares;
        if (A[j] <= x) {                // 4  <= not < : keeps the
invariant on ties
            i = i + 1;                  // 5
            ++partitionSwaps;
            swap(A[i], A[j]);           // 6  grow the low side by
one
        }
    }
    ++partitionSwaps;
    swap(A[i + 1], A[r]);               // 7  drop the pivot
between the two sides
    return i + 1;                       // 8  its FINAL index -- it
never moves again
}

void quicksort(vector<int>& A, int p, int r) {
    if (p < r) {                                // 1
        int q = partitionLomuto(A, p, r);       // 2
        quicksort(A, p, q - 1);                 // 3  note: q is
EXCLUDED from both
        quicksort(A, q + 1, r);                 // 4  calls -- it
is already correct
```

```
    }
}
```

Complexity. `PARTITION` is `Θ(r − p)` — one linear scan. Quicksort:

- **Worst case `Θ(n²)`**, when every partition is maximally unbalanced: `T(n) = T(n−1) + Θ(n)`.
- **Best/average `Θ(n lg n)`**. Even a fixed 9-to-1 split gives `T(n) = T(9n/10) + T(n/10) + Θ(n) = Θ(n lg n)` — the recursion tree just has depth `log_{10/9} n` instead of `lg n`.
- **Space `Θ(lg n)` expected stack**, `Θ(n)` worst case. Recurse on the smaller side first and loop on the larger to force `Θ(lg n)`.

The worst case is an already-sorted array, which is the single most common real-world input. That is not a corner case; that is Tuesday.

> *Verified, on an **already-sorted** array of `n = 4000`:*
>
> | *PIVOT RULE* | *COMPARISONS* | *PREDICTED* |
> |---|---|---|
> | *last element (deterministic)* | ***7 998 000*** | *`n²/2` = 8 000 000* |
> | *random (`A6`)* | ***57 085*** | *`2n ln n` = 66 352* |
>
> ***140× fewer comparisons**, and the randomized version's recursion depth was 27 against `lg 4000 ≈ 12`. Both sorted correctly; only one of them finished quickly.*

A6 RANDOMIZED-PARTITION

Pseudocode: §4, "Randomized quicksort".

```
// Three lines that convert a worst case over INPUTS into an
expectation over
// COIN FLIPS. There is no longer any input an adversary can hand
you that is
// reliably bad -- the bad case now depends on the random draws,
which they
// cannot see.
int randomizedPartition(vector<int>& A, int p, int r) {
    int i = randomInt(p, r);        // 1  RANDOM(p, r) -- INCLUSIVE
both ends
```

```
    swap(A[r], A[i]);                // 2  move the chosen pivot to
the end...
    return partitionLomuto(A, p, r);// 3  ...so PARTITION is reused
UNCHANGED
}

long long quicksortDepth = 0;

// The default argument tracks recursion depth for the measurement
below; it is
// not part of the algorithm.
void randomizedQuicksort(vector<int>& A, int p, int r, long long
depth = 1) {
    quicksortDepth = max(quicksortDepth, depth);
    if (p < r) {
        int q = randomizedPartition(A, p, r);
        randomizedQuicksort(A, p, q - 1, depth + 1);
        randomizedQuicksort(A, q + 1, r, depth + 1);
    }
}
```

Complexity. $O(n \lg n)$ **expected**, for **every** input — the expectation is over the algorithm's own randomness, not over a distribution of inputs. The worst case is still $\theta(n^2)$, but it now requires an astronomically unlucky sequence of draws rather than a sorted array.

Expected comparisons $\approx 2n \ln n \approx 1.39\, n \lg n$. The proof is the indicator-variable argument of M04: z_i and z_j are compared at most once, exactly when one of them is the first of $\{z_i, \ldots, z_j\}$ to be chosen as a pivot, which has probability $2/(j - i + 1)$.

> *Verified: 57 085 comparisons on the sorted $n = 4000$ input against the predicted 66 352 — see the table in A5.*

A7 COUNTING-SORT

Pseudocode: §6, "Algorithm: Counting Sort".

```
// Requires keys in [0, k]. Returns a NEW array -- counting sort is
not in place.
vector<int> countingSort(const vector<int>& A, int k) {
    const int n = (int)A.size() - 1;
```

```
    vector<int> B(n + 1);                          // output
    vector<int> C(k + 1, 0);                       // C[i] will count
value i

    for (int j = 1; j <= n; ++j) C[A[j]] = C[A[j]] + 1;   //
histogram
    for (int i = 1; i <= k; ++i)  C[i] = C[i] + C[i - 1]; // PREFIX
SUMS:
                                   // C[i] is now "how many values
are <= i",
                                   // i.e. the last output slot
value i may occupy

    for (int j = n; j >= 1; --j) {             // <-- REVERSE
order. See below.
        B[C[A[j]]] = A[j];
        C[A[j]] = C[A[j]] - 1;                 // next equal key
goes one slot earlier
    }
    return B;
}

struct Rec { int key; int id; };               // id = original
position, to test stability

vector<Rec> countingSortStable(const vector<Rec>& A, int k) {
    const int n = (int)A.size() - 1;
    vector<Rec> B(n + 1);
    vector<int> C(k + 1, 0);
    for (int j = 1; j <= n; ++j) C[A[j].key]++;
    for (int i = 1; i <= k; ++i)  C[i] += C[i - 1];
    for (int j = n; j >= 1; --j) { B[C[A[j].key]] = A[j];
C[A[j].key]--; }
    return B;
}
```

Complexity. $\Theta(n + k)$ time, $\Theta(n + k)$ space. **Linear when $k = O(n)$** — and useless when k is large: sorting 1000 32-bit integers this way needs a 4×10^9-element counter array.

No comparisons are performed, which is why the `Ω(n lg n)` decision-tree lower bound does not apply.

The reversed final loop is the entire stability argument. Walking `j` from `n` down to `1` while decrementing `C` places the *last* occurrence of a key in the *last* slot available for that key — so equal elements come out in their original relative order. Walk it forward and the order of equal elements is **reversed**. And stability is not a nicety here: `RADIX-SORT` is built on it and is simply wrong without it.

> *Verified: matches `std::sort` on 400 arrays. On 500 records with 9 distinct keys, the reverse-loop version is **stable** (equal keys emerge with increasing original index); the identical code with a **forward** final loop was measured to invert the order of equal keys — the bug is real, not theoretical.*

A8 RADIX-SORT

Pseudocode: §6, "Algorithm: Radix Sort".

```
// d passes of a STABLE sort, LEAST significant digit first.
// A is taken BY VALUE because each pass overwrites it.
vector<int> radixSort(vector<int> A, int d, int base = 10) {
    const int n = (int)A.size() - 1;
    vector<int> B(n + 1);
    long long place = 1;                          // 1, base,
base^2, ...  long long: base^d
                                                  // overflows int
for d around 10
    for (int pass = 1; pass <= d; ++pass) {       // 1  for i = 1 to
d
        // 2  "use a STABLE sort on digit i" -- counting sort,
inlined
        vector<int> C(base, 0);
        for (int j = 1; j <= n; ++j) C[(A[j] / place) % base]++;
        for (int i = 1; i < base; ++i) C[i] += C[i - 1];
        for (int j = n; j >= 1; --j) {            // reversed:
STABILITY, see A7
            int digit = (int)((A[j] / place) % base);
            B[C[digit]] = A[j];
            C[digit]--;
        }
        A = B;                                    // copy back; a
swap() would avoid the copy
```

```
        place *= base;
    }
    return A;
}
```

Complexity. `Θ(d(n + base))`. With `base = n` and `b`-bit keys, `d = ⌈b / lg n⌉` and the total is `Θ(n · b/lg n)` — **`Θ(n)` for fixed-width keys**.

Why LSD, and why stability is load-bearing. After sorting on digit `i`, the array is correctly ordered by digits `1..i`. Sorting on digit `i+1` must **not disturb** that order among elements whose digit `i+1` is equal — which is precisely stability. Use a non-stable sort per pass and radix sort silently returns garbage.

Is it faster than quicksort? Not automatically. Skiena's and CLRS's shared verdict: radix sort touches every bit of every key `d` times with poor locality, while quicksort's partition is a cache-friendly linear scan. Radix wins on large `n` with short keys; measure before switching.

> *Verified: `d = 6` decimal digits, keys in `[0, 999999]` — matches `std::sort` on 300 arrays.*

A9 BUCKET-SORT

Pseudocode: §6, "Algorithm: Bucket Sort".

```
void insertionSortList(vector<double>& v) {
    for (size_t i = 1; i < v.size(); ++i) {
        double key = v[i];
        size_t j = i;
        // `j > 0` first, then v[j-1]: with SIZE_T indices, j-1 at
j==0 wraps to
        // SIZE_MAX and reads far out of bounds. Short-circuiting
saves us.
        while (j > 0 && v[j - 1] > key) { v[j] = v[j - 1]; --j; }
        v[j] = key;
    }
}

// Assumes the input is drawn UNIFORMLY from [0, 1). That
assumption is the
// entire analysis -- it is what makes the buckets evenly filled.
```

```cpp
vector<double> bucketSort(const vector<double>& A) {
    const int n = (int)A.size() - 1;
    vector<vector<double>> B(n);                    // 1  n empty
lists
    for (int i = 1; i <= n; ++i) {
        int idx = (int)(n * A[i]);                  // 4  bucket
floor(n * A[i])
        if (idx >= n) idx = n - 1;                  // guard: A[i] ==
1.0 would index n
        B[idx].push_back(A[i]);
    }
    vector<double> out;
    out.reserve(n + 1);
    out.push_back(0.0);                             // keep the 1-
based convention
    for (int i = 0; i < n; ++i) {
        insertionSortList(B[i]);                    // 6  sort each
bucket
        for (double x : B[i]) out.push_back(x);     // 8  concatenate
in order
    }
    return out;
}
```

Complexity. $\Theta(n)$ **expected** under the uniform-input assumption, $\Theta(n^2)$ worst case (everything in one bucket). The analysis is $E[\Sigma\, n_i^2] = 2n - 1$: bucket sizes are $\text{Binomial}(n, 1/n)$, so $E[n_i^2] = \text{Var} + \text{mean}^2 = (1 - 1/n) + 1 = 2 - 1/n$, and summing over n buckets gives $2n - 1$.

This is an average-case bound that depends on the input distribution — unlike randomized quicksort, whose expectation comes from the algorithm's own coins. Feed `bucketSort` a non-uniform distribution and it degrades, and randomizing will not save it. (Skiena's fix: choose bucket boundaries from a *sample* of the data.)

> *Verified: matches* `std::sort` *on 300 uniform arrays. At* $n = 100\,000$ *uniform inputs the* ***fullest bucket held 7 elements****, and* $\Sigma\, n_i^2 = 200\,382$ *against the predicted* $2n - 1 = 199\,999$ *— agreement to 0.2%.*

A10 RANDOMIZED-SELECT

Pseudocode: §7, "Algorithm: RANDOMIZED-SELECT".

```
// Returns the i-th SMALLEST element of A[p..r], with i counted
from 1.
// Same partition as quicksort -- but recurses into only ONE side,
which is the
// entire reason the cost drops from n lg n to n.
int randomizedSelect(vector<int>& A, int p, int r, int i) {
    if (p == r) return A[p];                        // 1  one
element: it is the answer
    int q = randomizedPartition(A, p, r);           // 3
    int k = q - p + 1;                              // 4  rank of
the pivot WITHIN A[p..r]
    if (i == k)      return A[q];                   // 5  the pivot
is the answer
    else if (i < k)  return randomizedSelect(A, p, q - 1, i);
 // 7  left side
    else             return randomizedSelect(A, q + 1, r, i - k);
 // 9  right side,
                     // and note `i - k`: the rank must be RE-
BASED, because the
                     // k elements at or before the pivot are gone.
Forgetting
                     // this is the classic quickselect bug.
}

// Both recursive calls are in tail position, so quickselect
becomes a loop --
// and that drops the space from O(lg n) expected to O(1).
int randomizedSelectIterative(vector<int>& A, int p, int r, int i)
{
    while (p != r) {
        int q = randomizedPartition(A, p, r);
        int k = q - p + 1;
        if (i == k) return A[q];
        if (i < k)  r = q - 1;                      // shrink to the
left side
        else      { p = q + 1; i -= k; }            // shrink right
AND re-base i
    }
    return A[p];
}
```

Complexity. $\Theta(n)$ expected, $\Theta(n^2)$ worst case. The intuition for linear: each level partitions and then discards one side, so the work is $n + n/2 + n/4 + \ldots = 2n$ when splits are balanced — a **geometric** series, not the n levels of n work that sorting pays. Formally $E[T(n)] \le E[T(3n/4)] + \Theta(n)$, master case 3, $\Theta(n)$.

The deterministic $\Theta(n)$ alternative is `SELECT` (median of medians): split into groups of 5, take the median of each, recursively select the median of those medians, use it as the pivot. It guarantees a 30/70 split, giving $T(n) \le T(n/5) + T(7n/10) + \Theta(n) = \Theta(n)$. The constant is bad enough that `RANDOMIZED-SELECT` wins in practice — median of medians is a *worst-case guarantee*, not a speedup.

In real code, use `std::nth_element` — it is this algorithm, with introselect's fallback for the worst case.

> *Verified: both forms return `sorted[i-1]` on 500 random arrays. At `n = 200 000`, selecting the median cost **1 157 759 comparisons (≈ 5.8 n)** while a full randomized quicksort of the same array cost **4 525 961 (≈ 1.3 n lg n)** — a 3.9× saving, which is the practical content of "selection is linear, sorting is not".*

Previous: M04 — Randomization & Probabilistic Analysis · Next: M06 — Elementary Data Structures